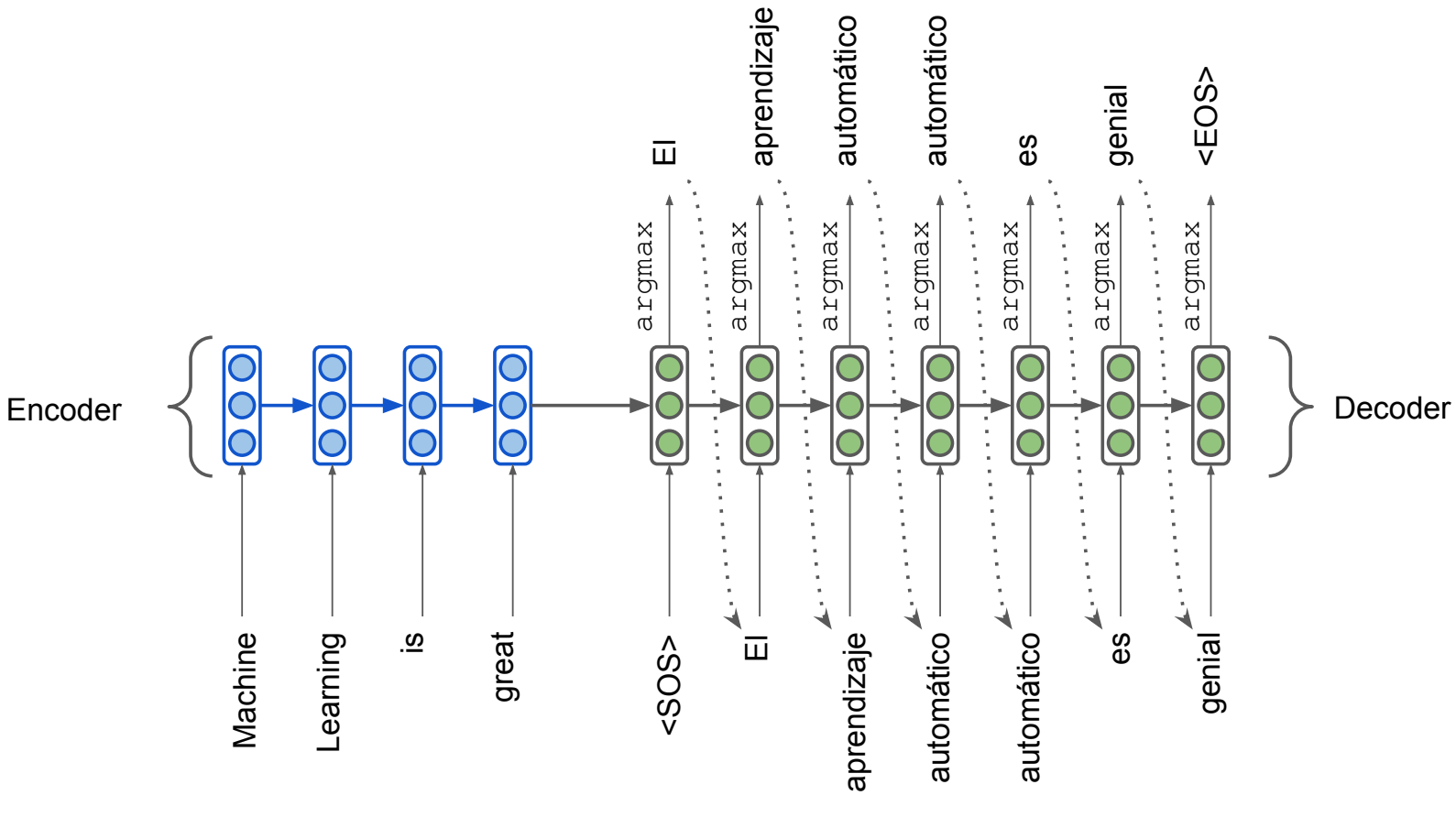
Attention mechanism. Transformer.

Nikolay Karpachev
11.03.2026

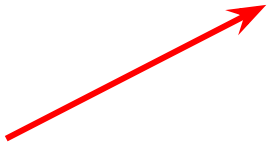
Attention

Sequence-to-Sequence Learning



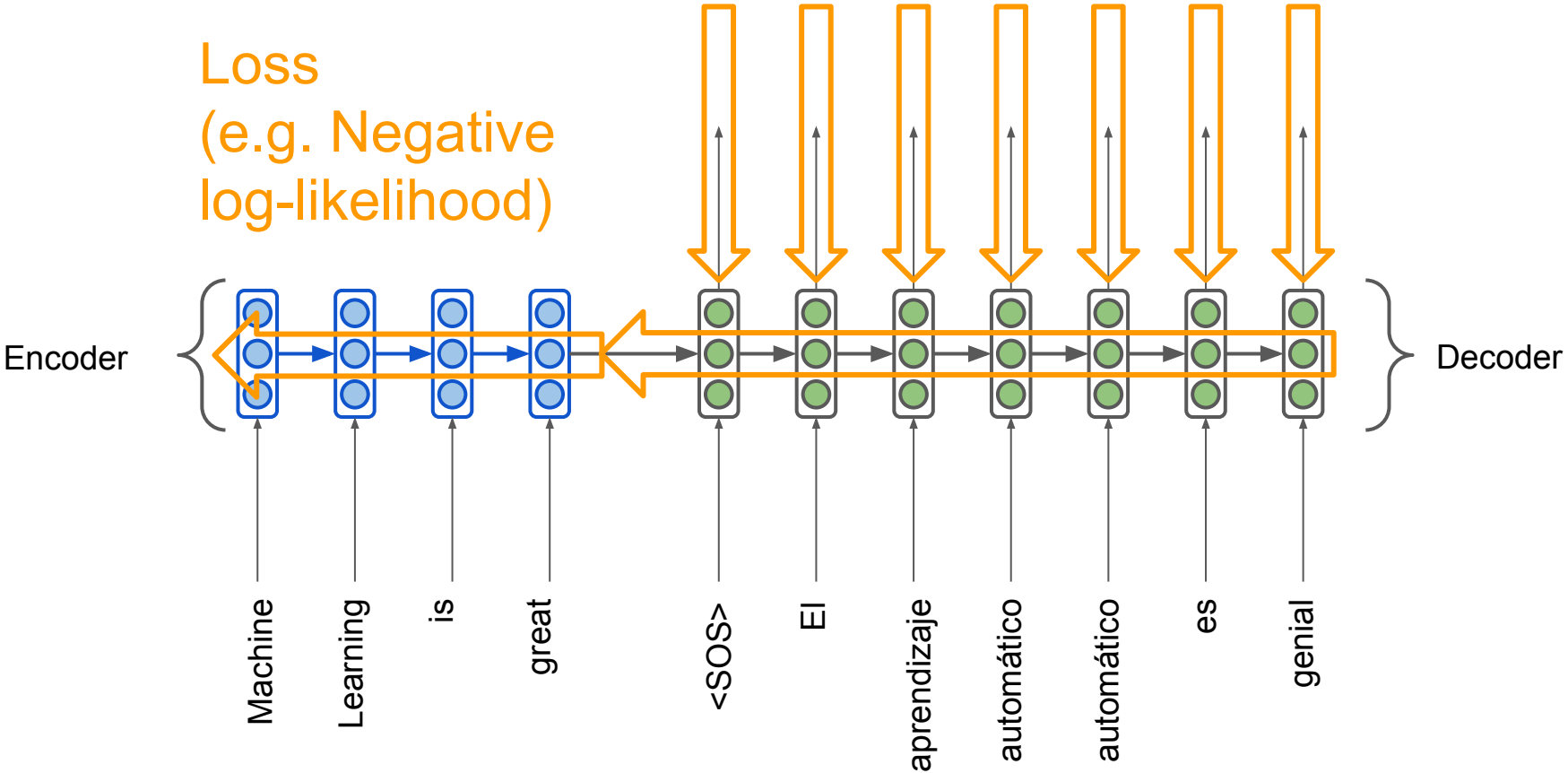
Sequence-to-Sequence Learning

$$P(y|x) = P(y_2|y_1, x)P(y_3|y_1, y_2, x)\ldots\underbrace{P(y_T|y_1, y_2, \ldots, x)}$$

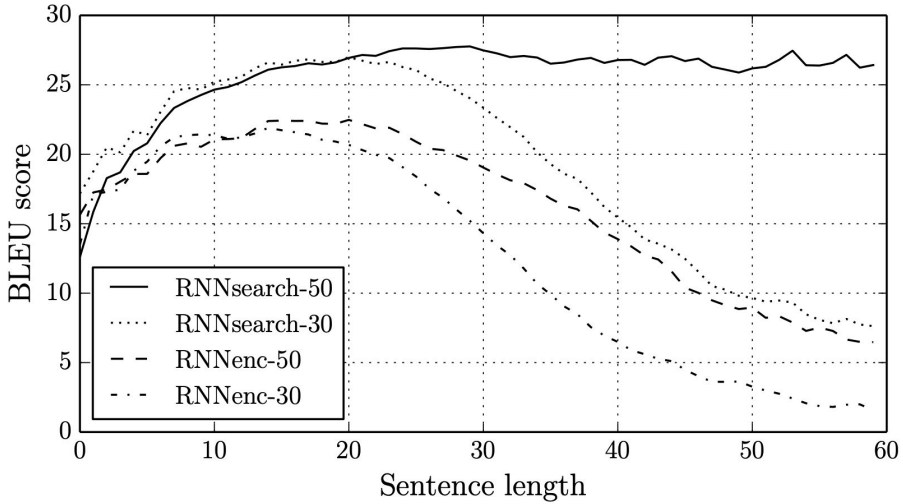


Probability of next word in target language

Seq2seq is trained end-to-end



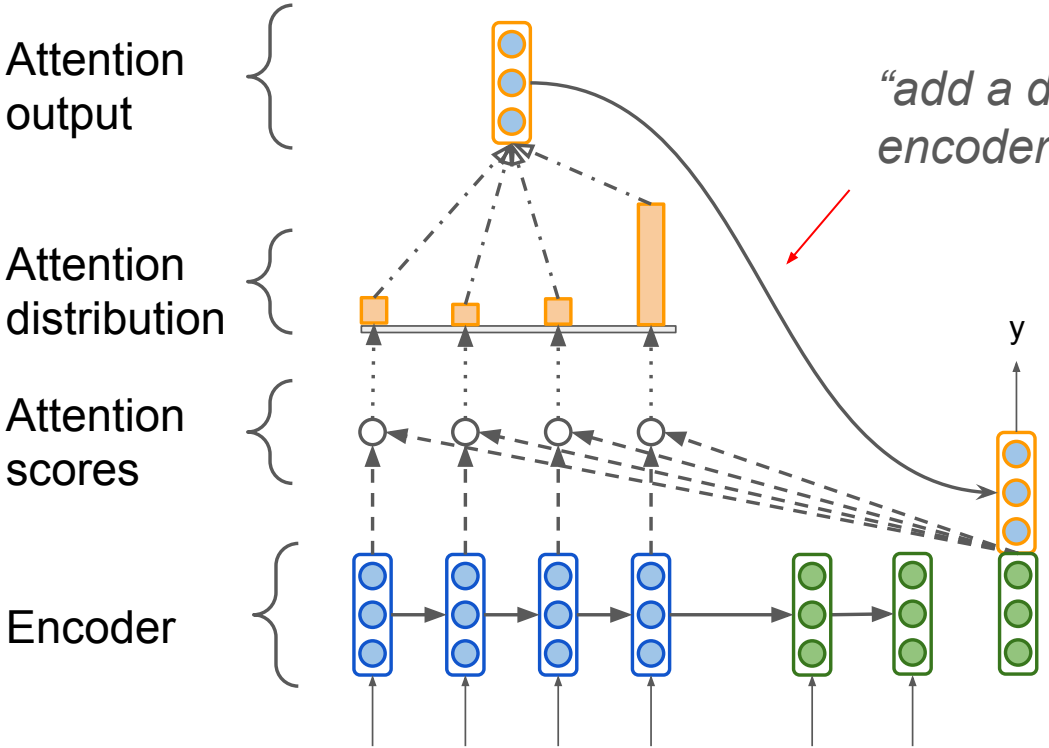
Attention (Bahdanau, 2014)



RNN for machine translation

- *RNNenc* – seq2seq
  - quality degrades with length
- *RNNsearch* – seq2seq + attention
  - much more stable

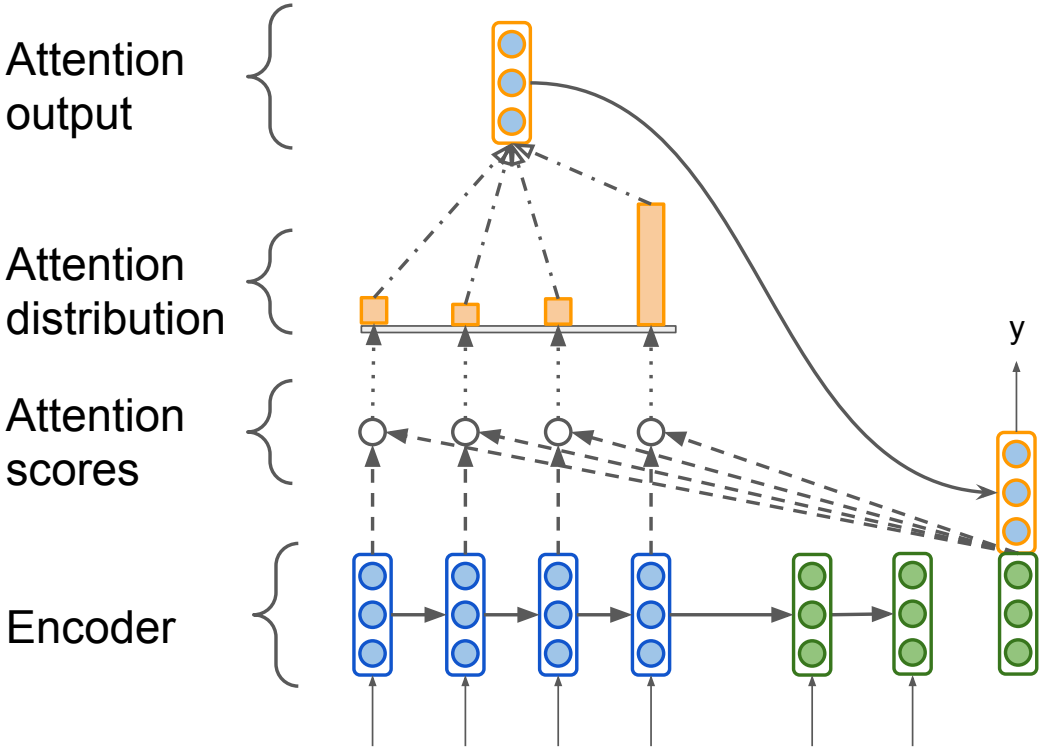
Seq2seq Attention



Main idea:

"add a direct connection between each encoder step and encoder hidden states"

Seq2seq Attention



Formally, attention is

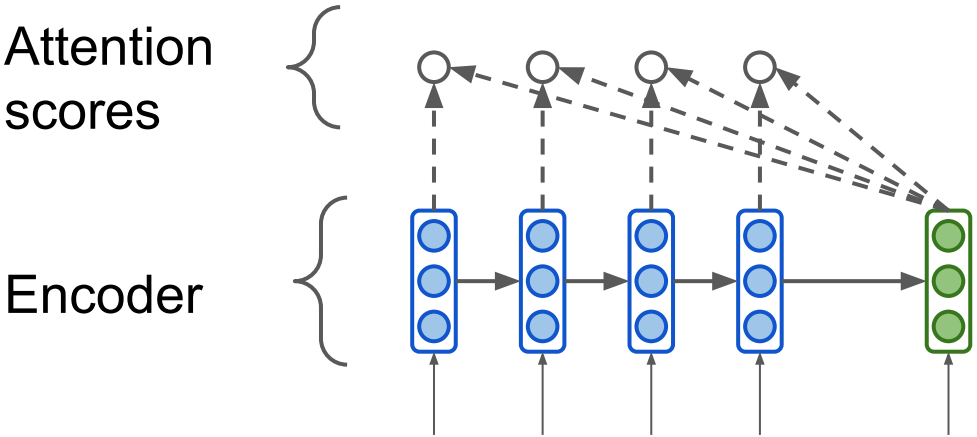
$$c_i = \sum_{j=1}^{T_x} \alpha_{ij} h_j$$

where

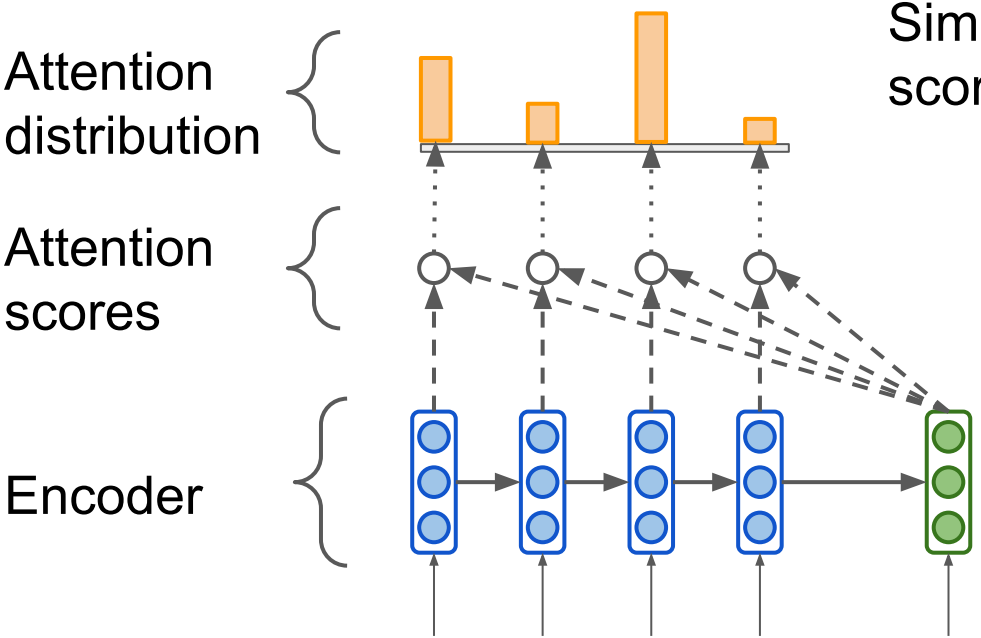
$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k=1}^{T_x} \exp(e_{ik})}$$

$$e_{ij} = a(s_{i-1}, h_j)$$

Seq2seq Attention

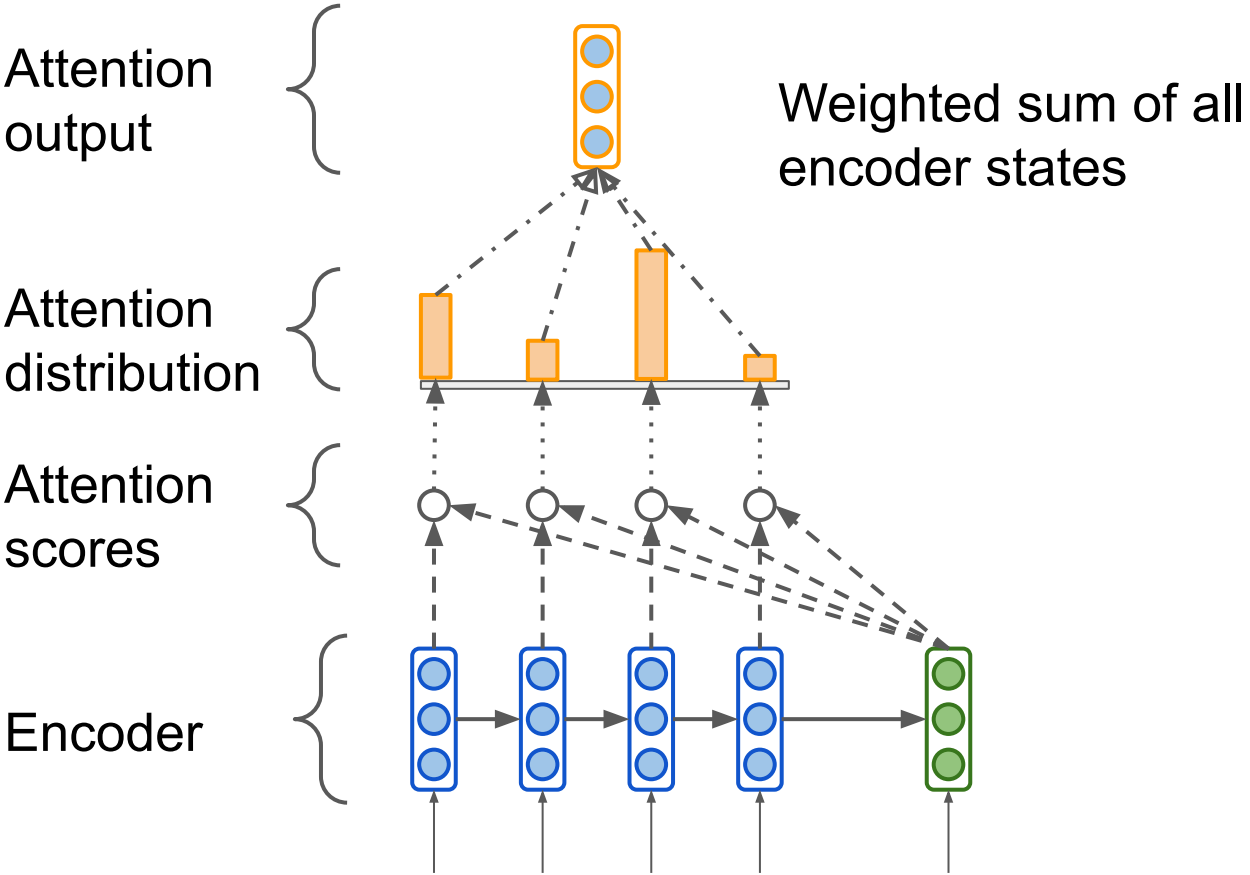


Seq2seq Attention

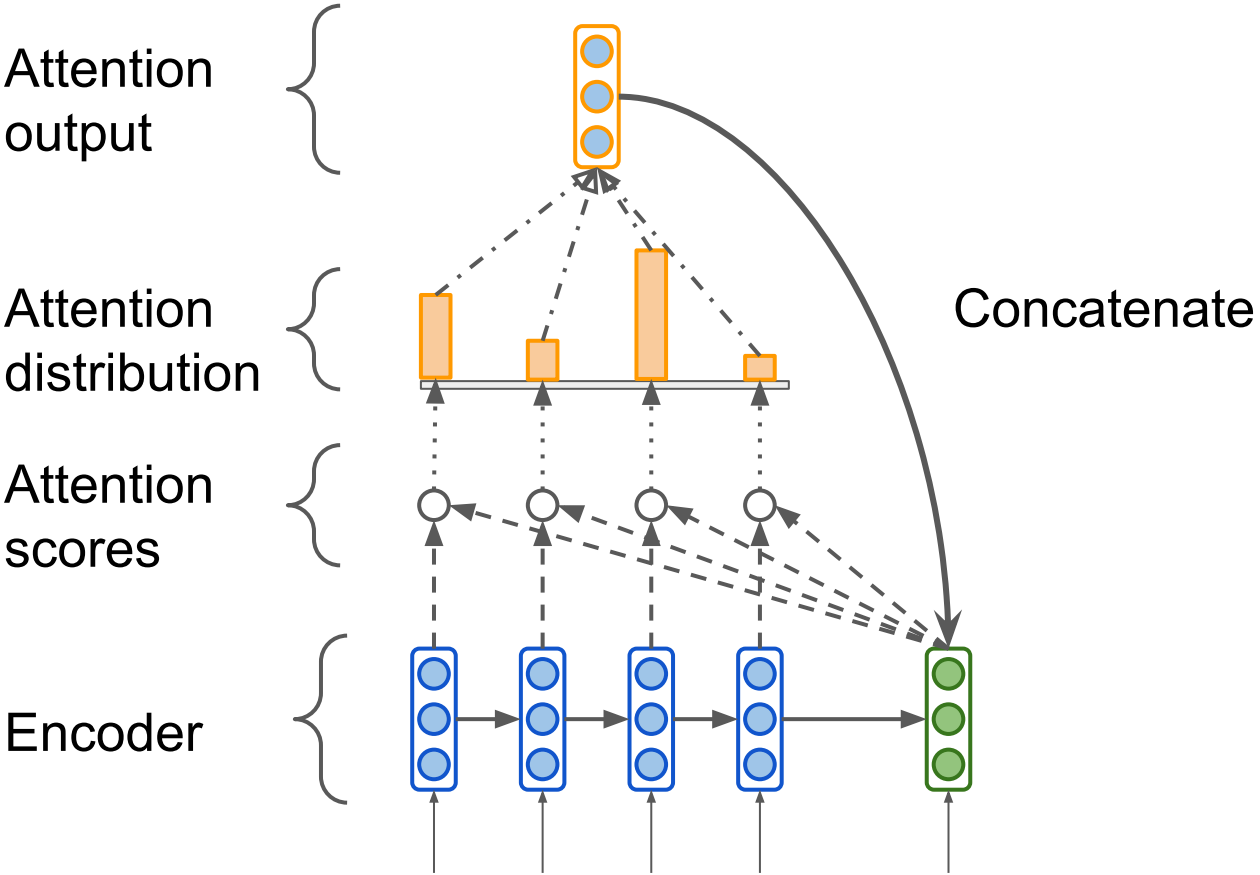


Simply apply softmax to scores

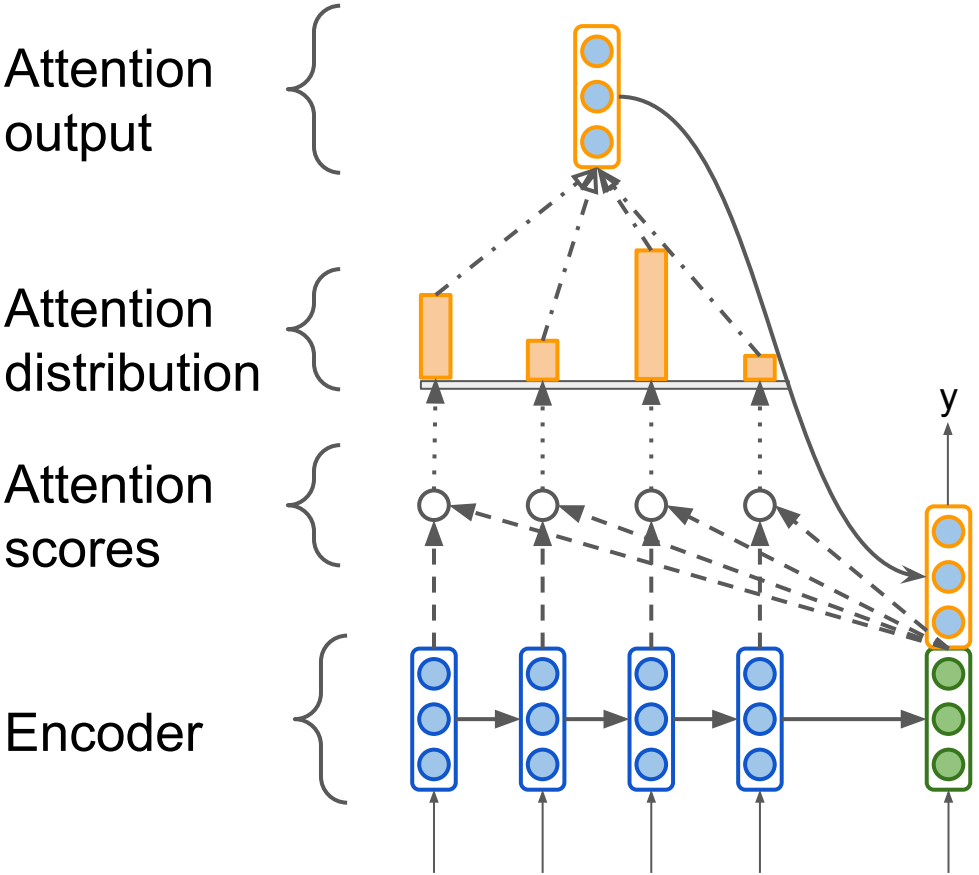
Seq2seq Attention



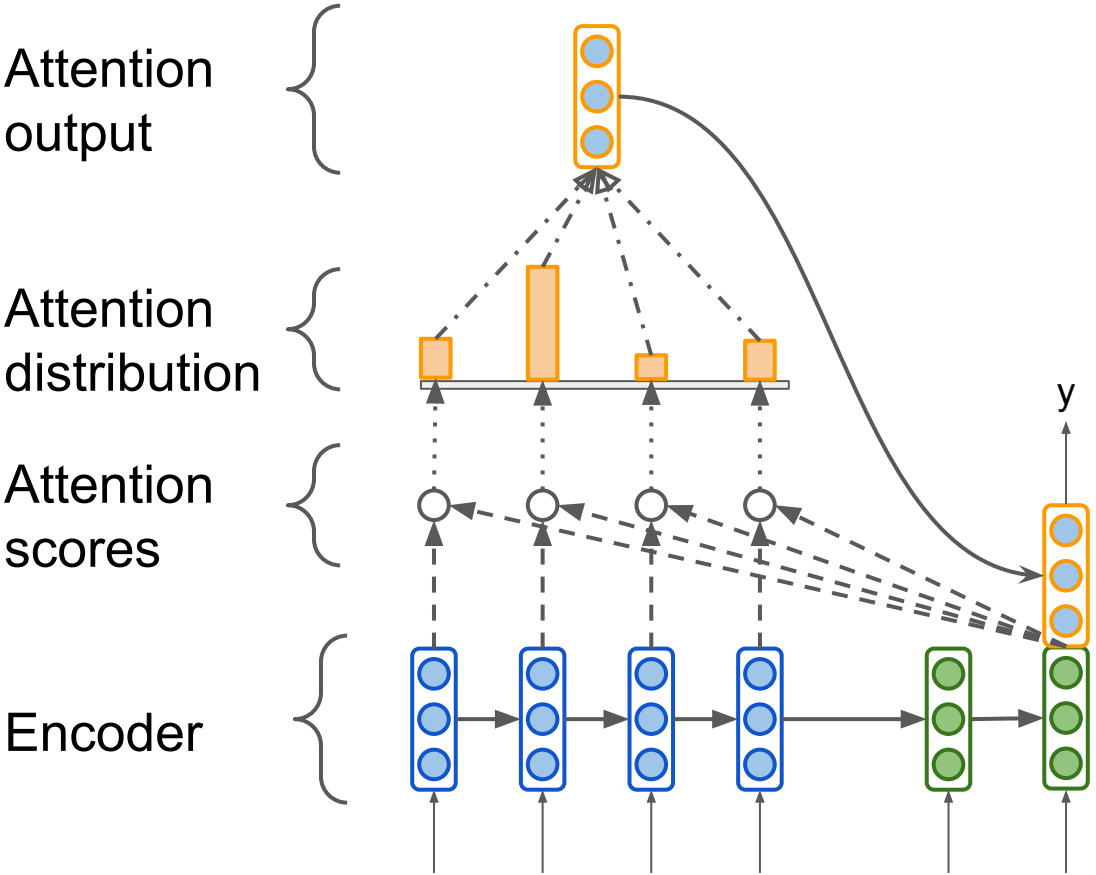
Seq2seq Attention



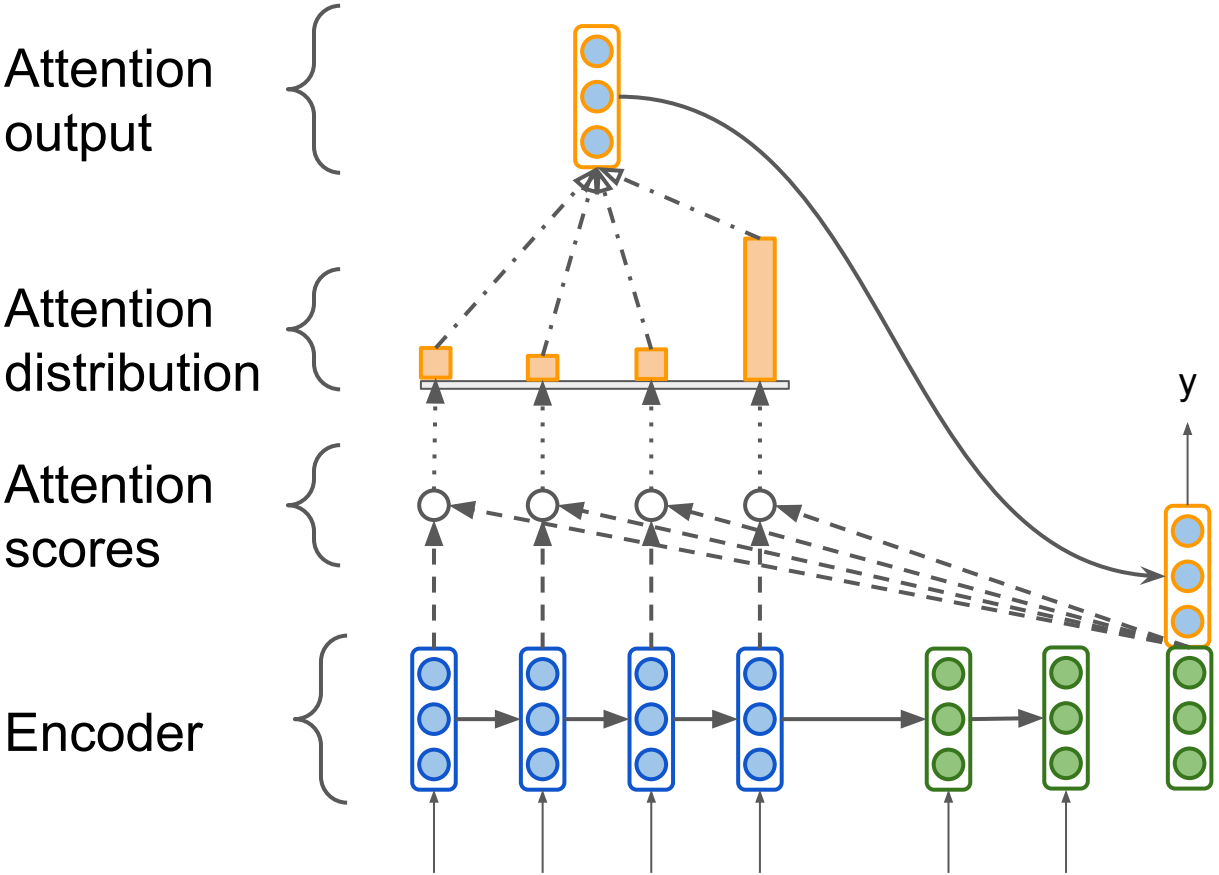
Seq2seq Attention



Seq2seq Attention

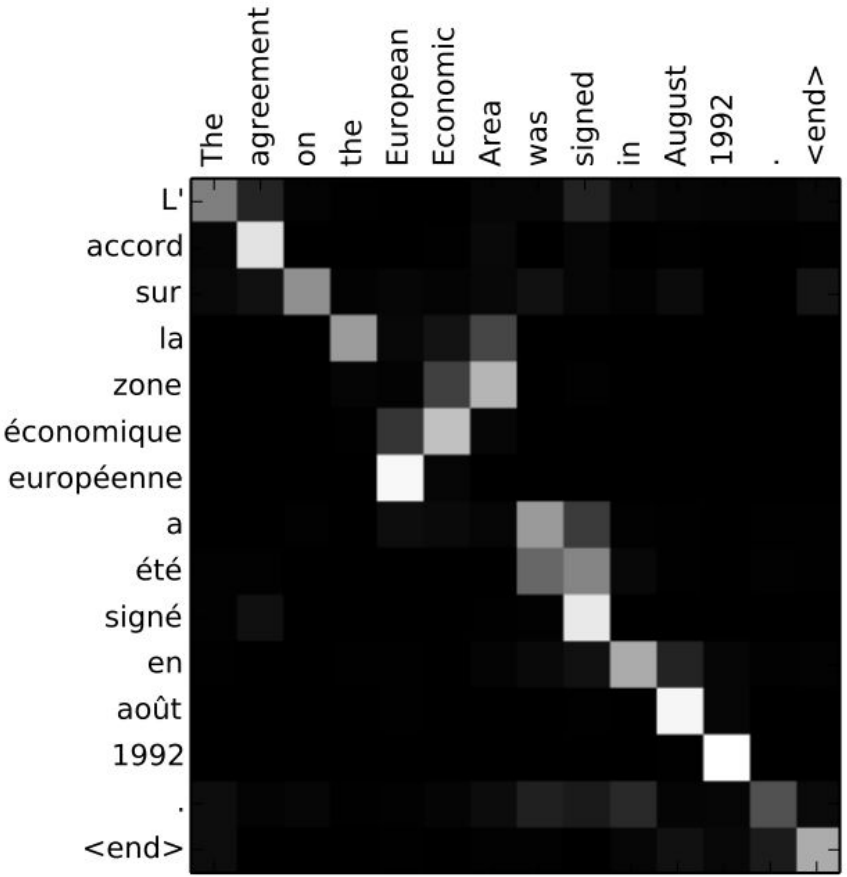


Seq2seq Attention



Attention provides interpretability

- We may see what the decoder was focusing on
- We get word alignment for free!



Bahdanau et al. "Neural Machine Translation by Jointly Learning to Align and Translate", 2014

Attention variants

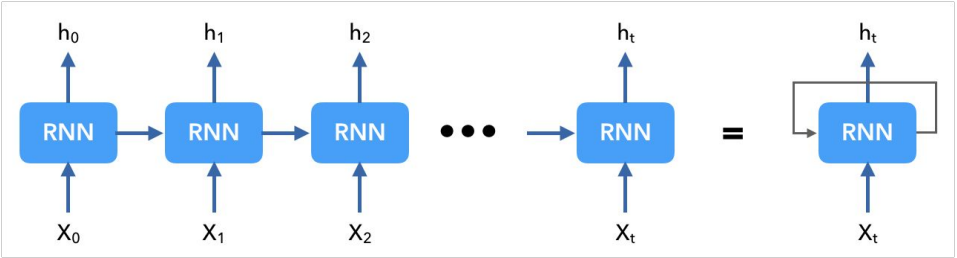
- Basic dot-product (the one discussed before): $e_i = s^T h_i \in \mathbb{R}$
- Multiplicative attention: $e_i = s^T W h_i \in \mathbb{R}$
  - $W \in \mathbb{R}^{d_2 \times d_1}$ - weight matrix
- Additive attention: $e_i = v^T \tanh(W_1 h_i + W_2 s) \in \mathbb{R}$
  - $W_1 \in \mathbb{R}^{d_3 \times d_1}, W_2 \in \mathbb{R}^{d_3 \times d_2}$ - weight matrices
  - $v \in \mathbb{R}^{d_3}$ - weight vector

Self-Attention

Self-Attention

Before Transformer

RNN / LSTM / GRU / ...



Sequence Processing is done sequentially

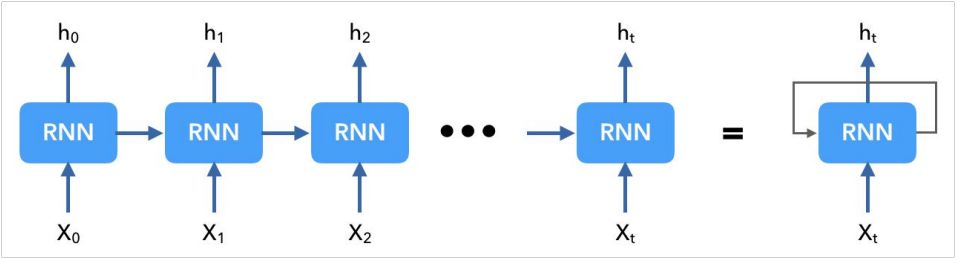
- vanishing gradients
- small influence of early tokens on the answer
- difficult inter-token dependency modeling

} *Linear interaction distance*

Self-Attention

Before Transformer

RNN / LSTM / GRU / ...



Self-attention motivation:

We would like a tool to model long contextual dependencies between tokens

Self-Attention at a High Level

”`The animal didn't cross the street because it was too tired`”

- What does “it” in this sentence refer to?

Self-Attention at a High Level

”`The animal didn't cross the street because it was too tired`”

- What does “it” in this sentence refer to?
- We want self-attention to associate “it” with “animal”

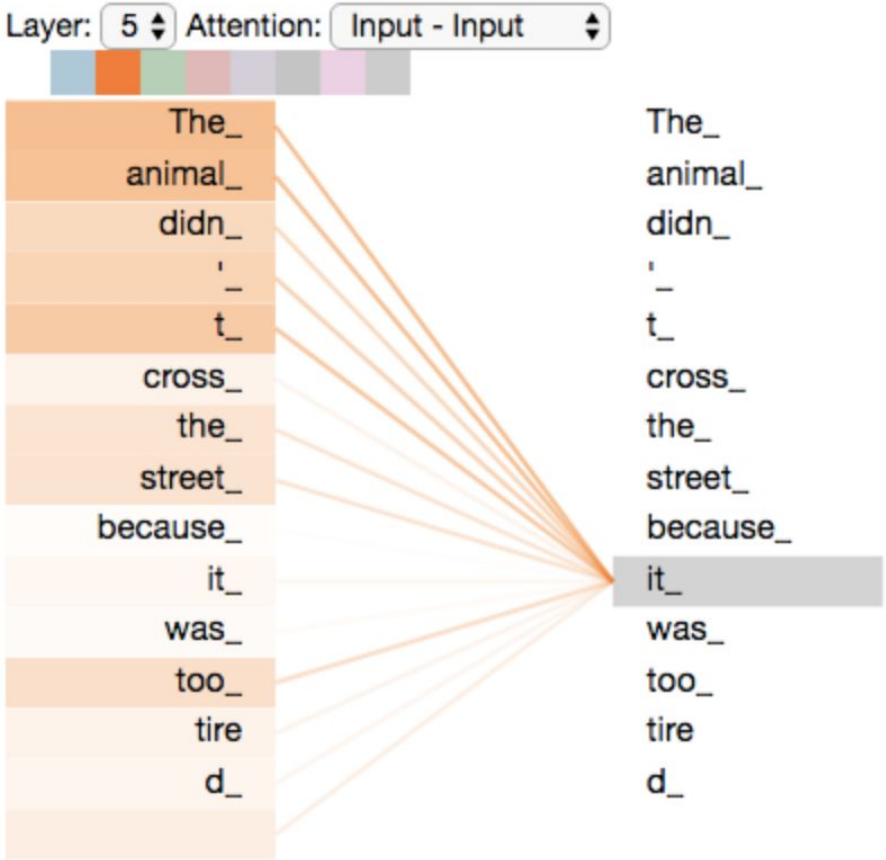
Self-Attention at a High Level

”`The animal didn't cross the street because it was too tired`”

- What does “it” in this sentence refer to?
- We want self-attention to associate “it” with “animal”

- Self-attention is the method the Transformer uses to bake the “understanding” of other relevant words into the one we’re currently processing

Self-Attention at a High Level



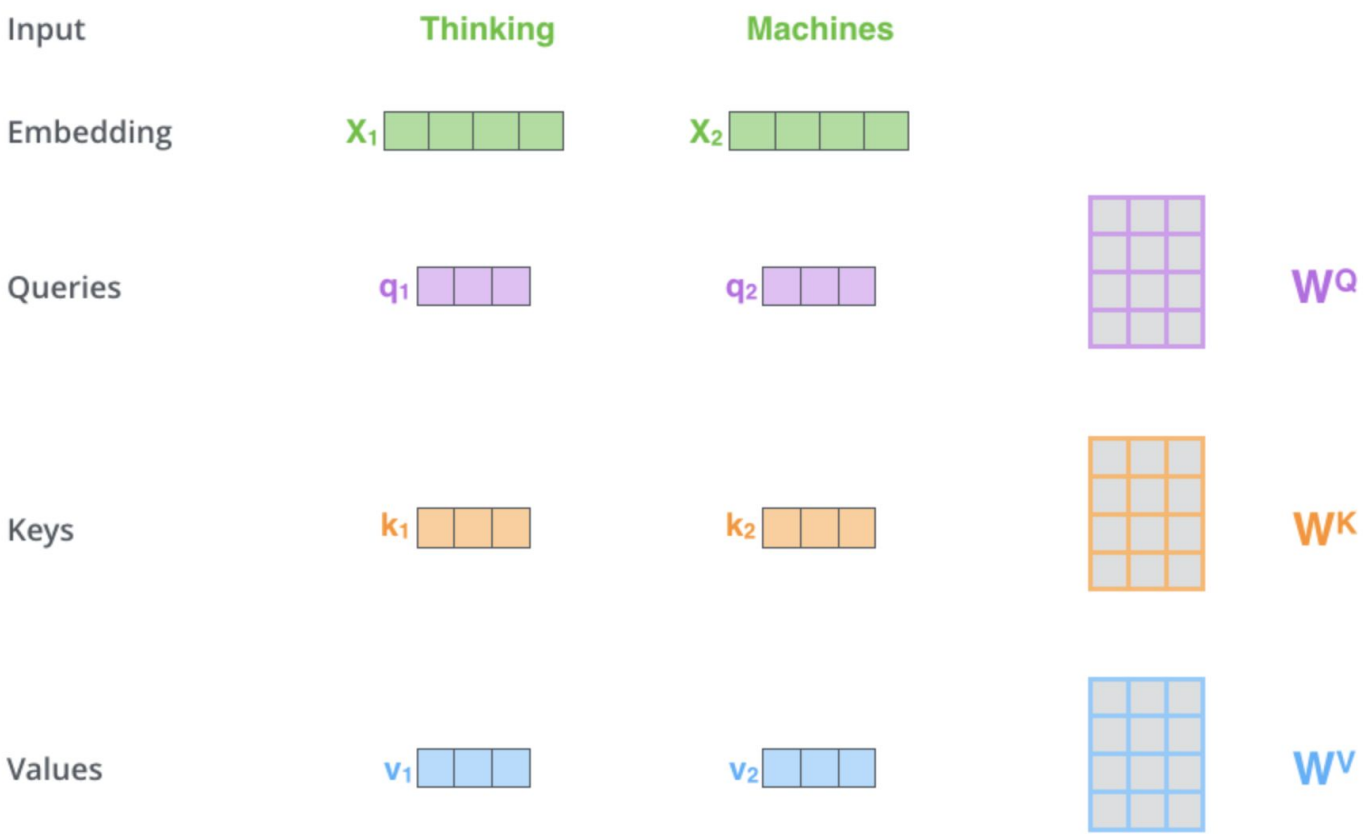
$$c_i = \sum_{j=1}^{T_x} \alpha_{ij} h_j$$

input embeddings for all tokens

contextual embedding for token i

"attention" is a weighed sum

Self-Attention: detailed explanation

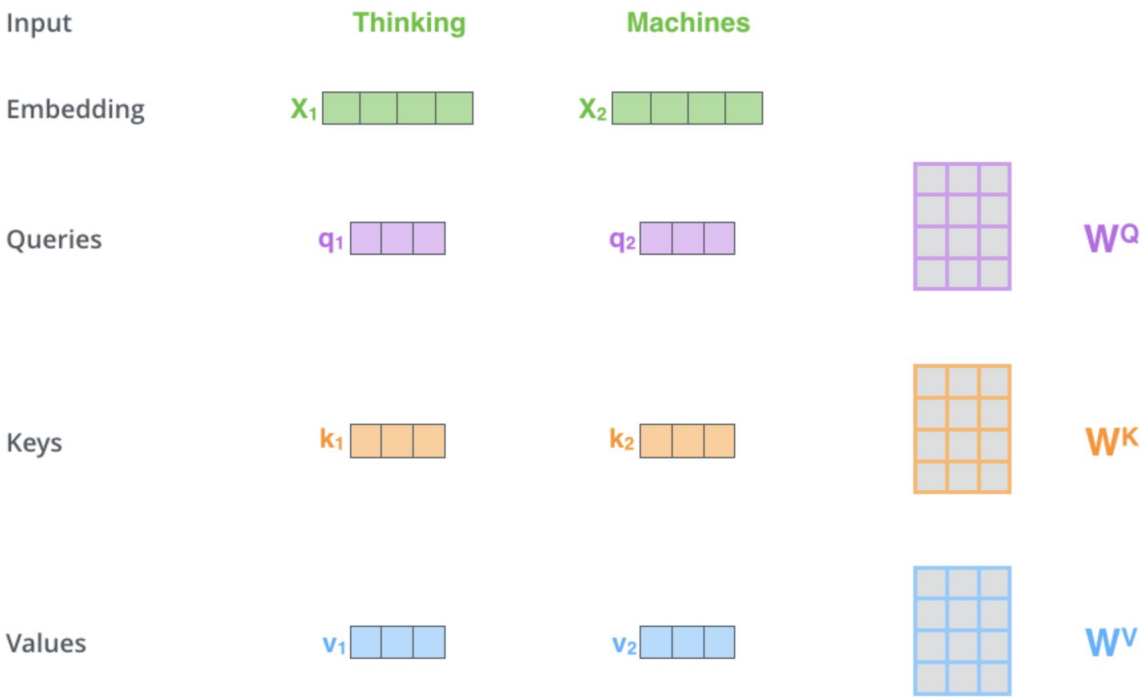


Self-Attention: detailed explanation

STEP 1:

create 3 vectors (Query, Key, Value)

from each of the encoder's input vectors



Self-Attention: detailed explanation

What are the “query”, “key”, and “value” vectors?

Self-Attention: detailed explanation

What are the “query”, “key”, and “value” vectors?

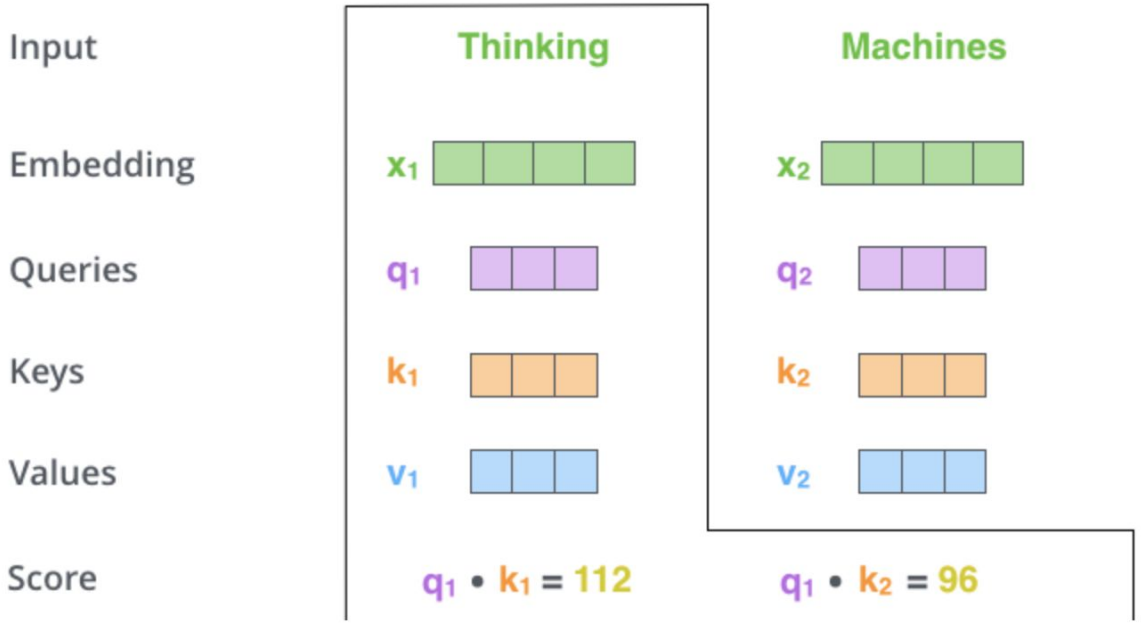
They’re abstractions that are useful for calculating and thinking about attention.

Self-Attention: detailed explanation

STEP 2:

calculate a score

(score each word of the input sentence against the current word)



Self-Attention: detailed explanation

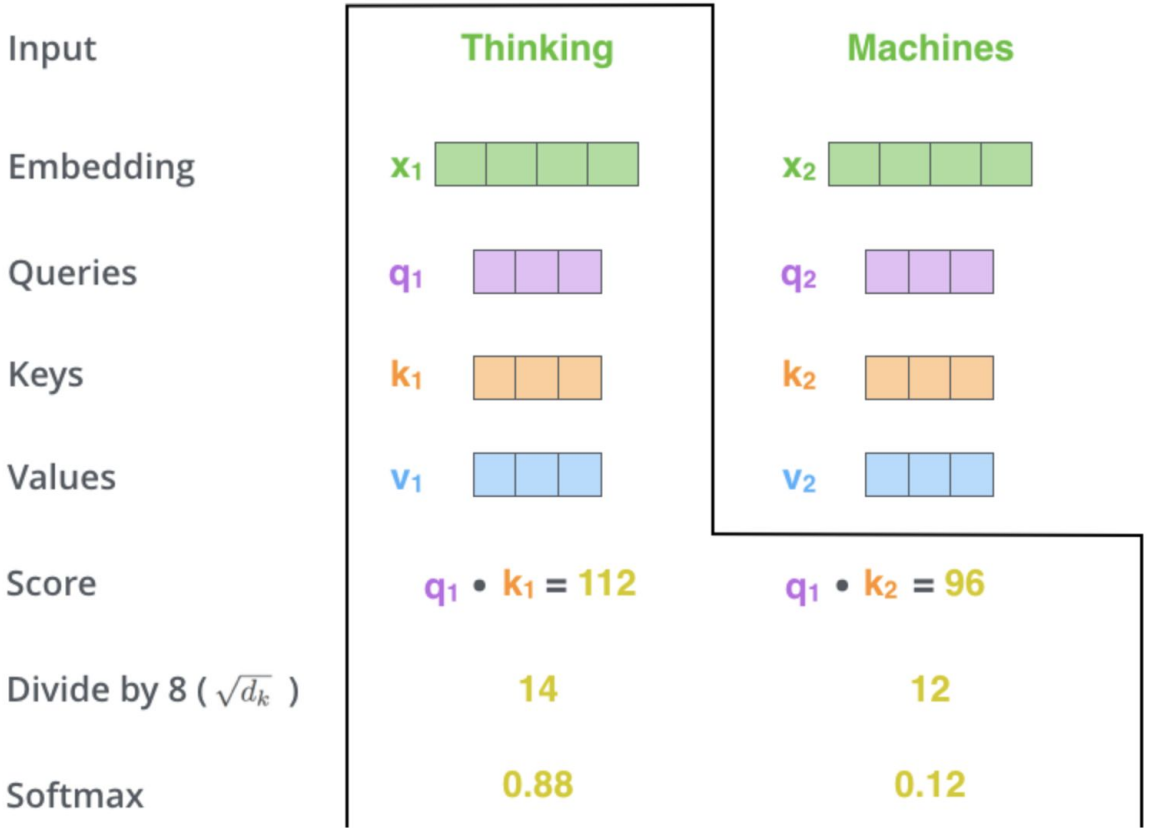
STEP 3:

divide the scores by 8

(the square root of the dimension of the key vectors)

STEP 4:

softmax



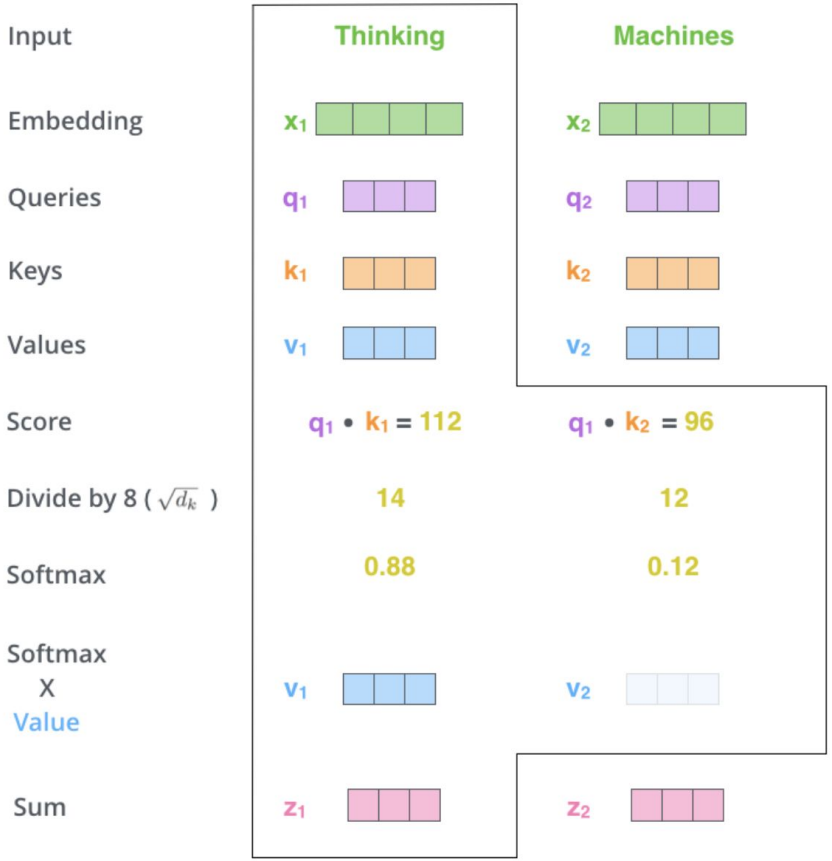
Self-Attention: detailed explanation

STEP 5:

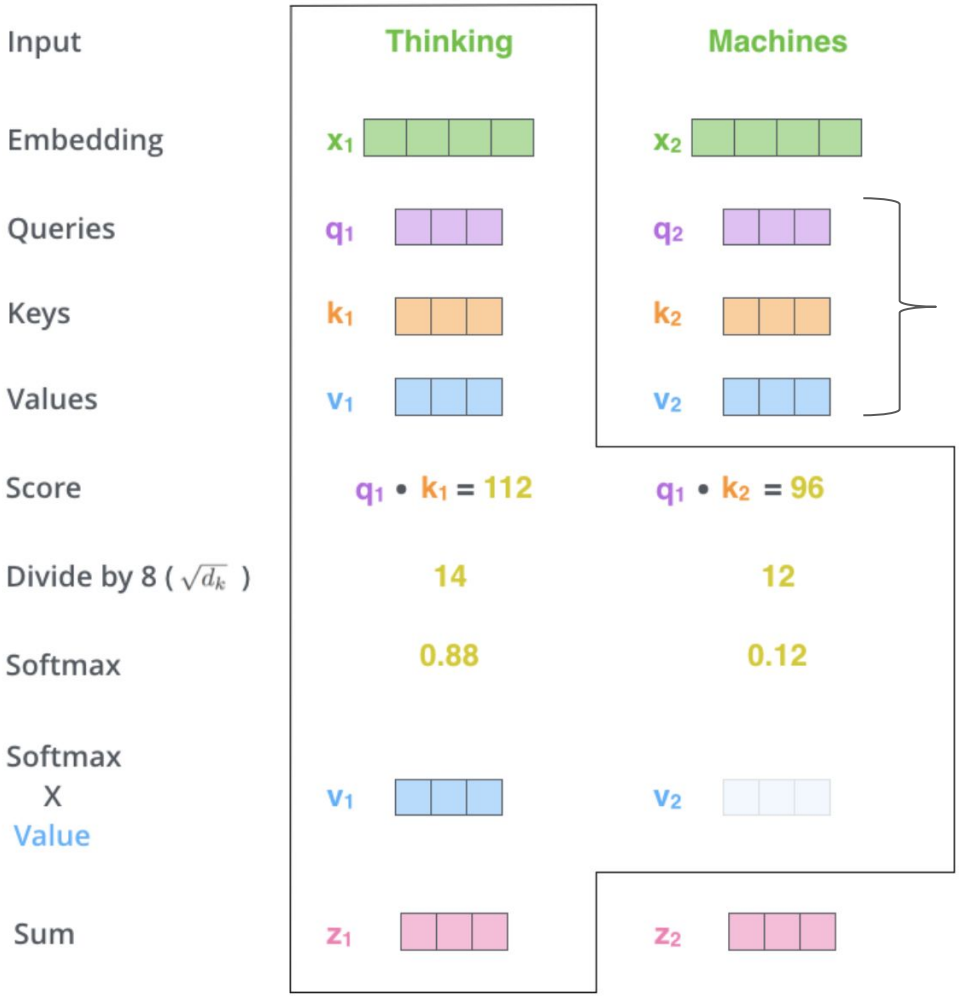
multiply each value vector by the softmax score

STEP 6:

sum up the weighted value vectors



Self-Attention



STEP 1: create Query, Key, Value

STEP 2: calculate scores

STEP 3: divide by $\sqrt{d_k}$

STEP 4: softmax

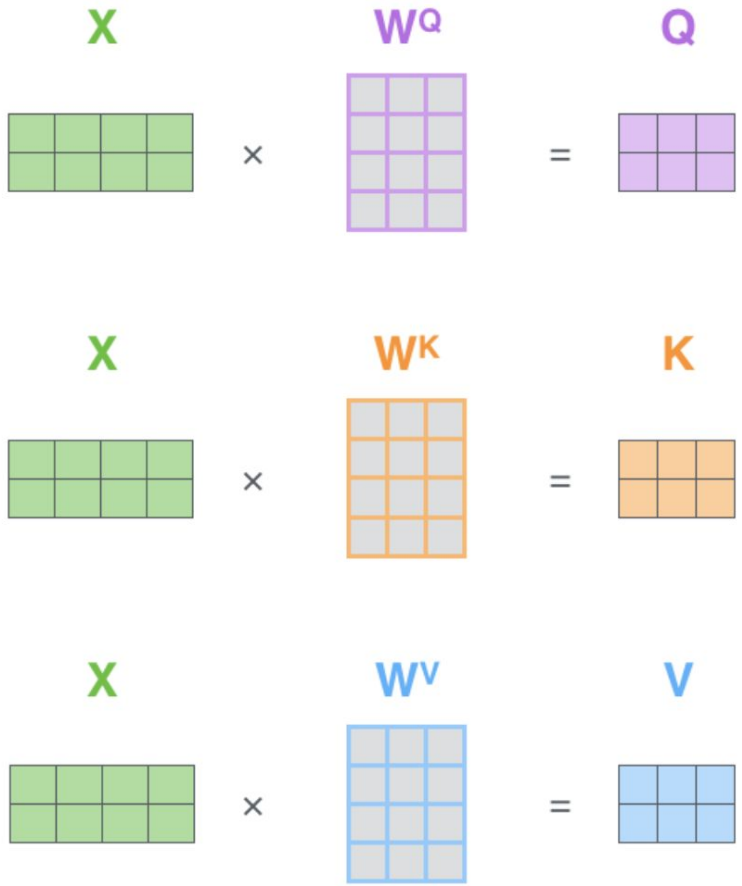
STEP 5: multiply each value vector by the softmax score

STEP 6: sum up the weighted value vectors

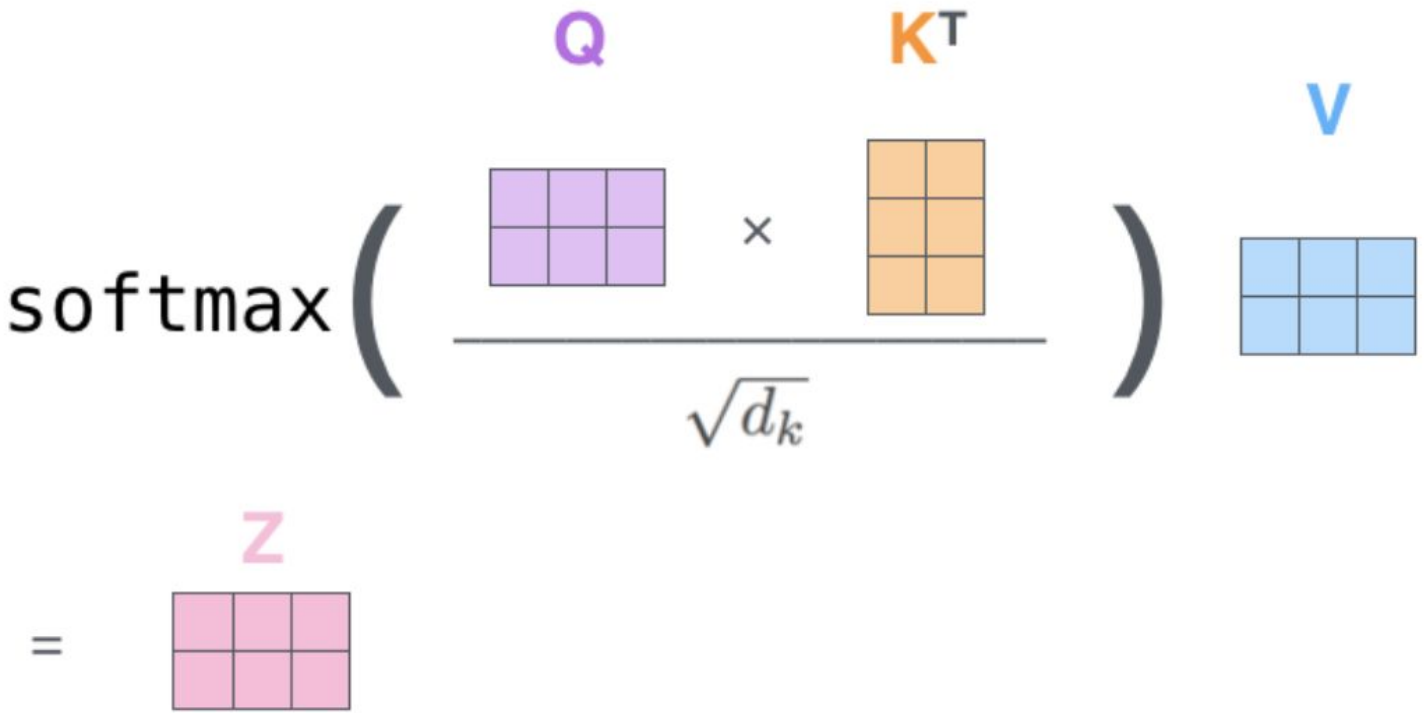
Self-Attention: Matrix Calculation

Pack embeddings into matrix **X**

Multiply **X** by weight matrices we've trained (**Wk, Wq, Wv**)



Self-Attention: Matrix Calculation



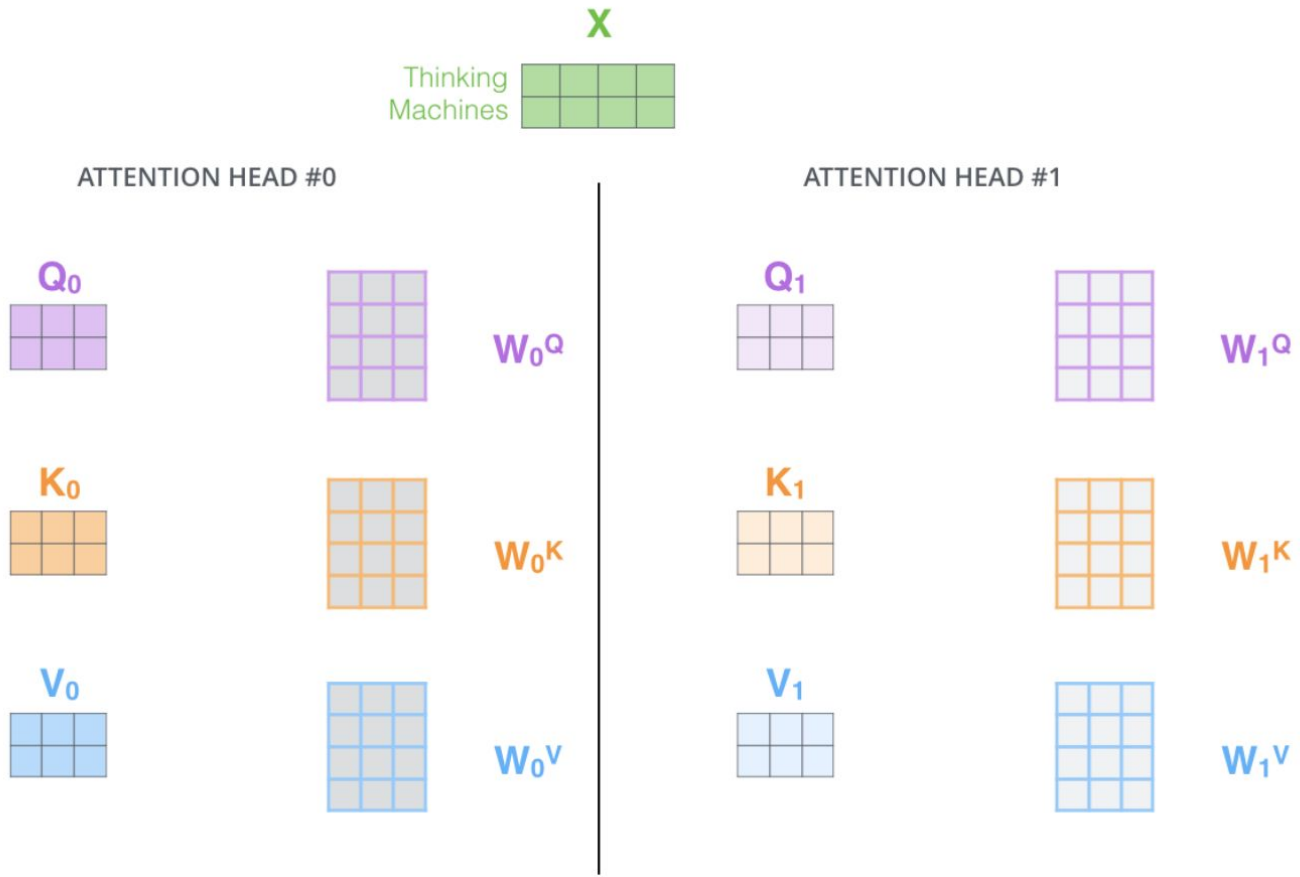
Multi-Head Attention

Attention – trainable weighed sum

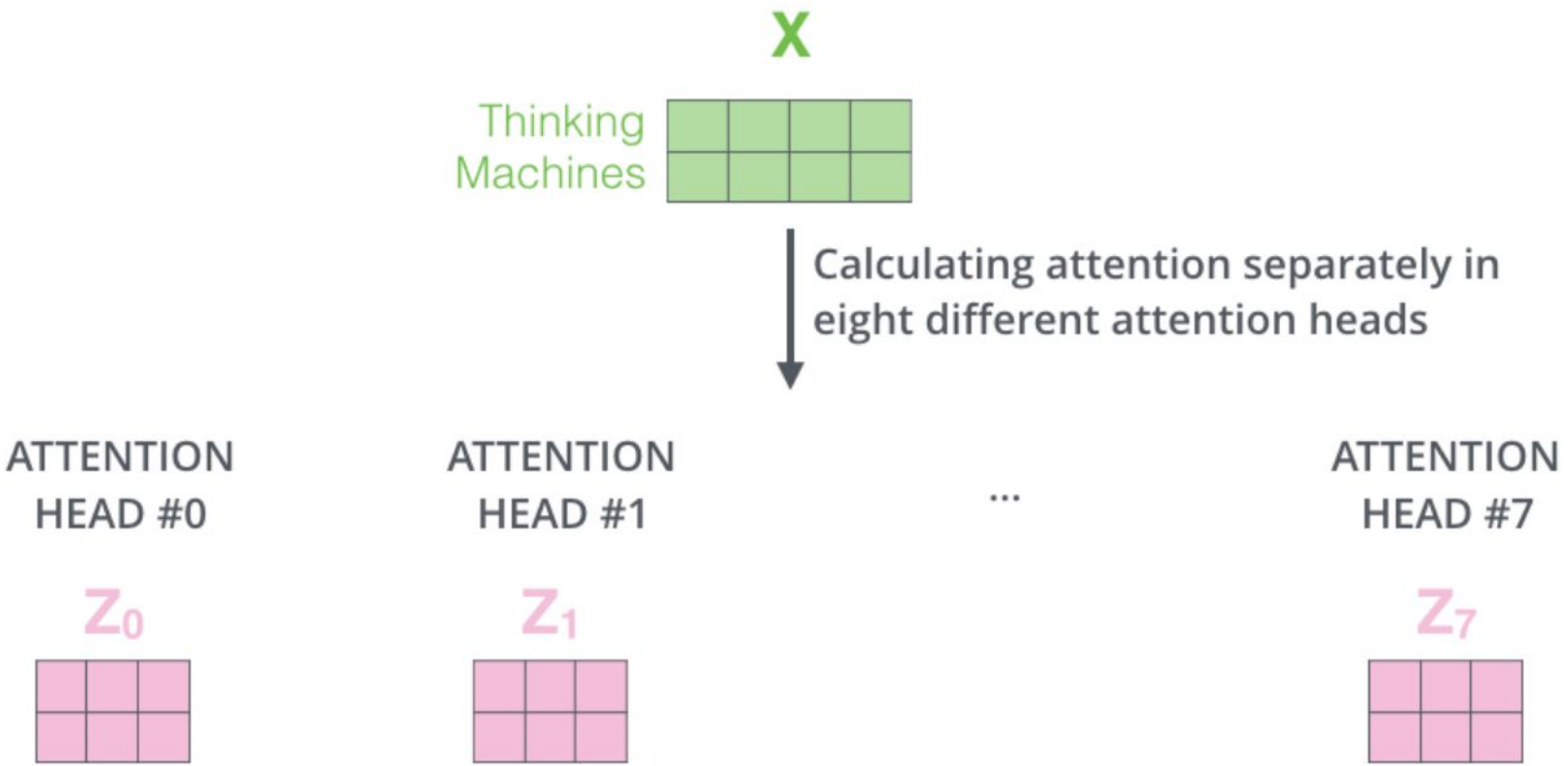
Multi-Head Attention – parallelized weighed sum with aggregation

Different “heads” attend to different nature of information

Multi-Head Attention



Multi-Head Attention

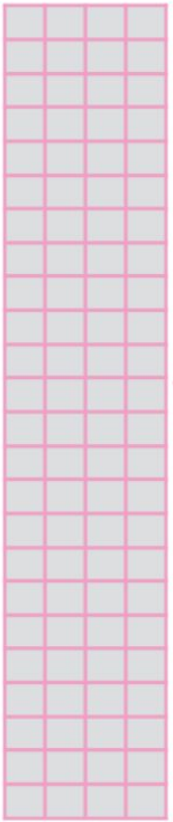


Multi-Head Attention

1) Concatenate all the attention heads

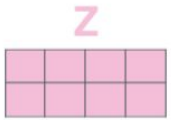
2) Multiply with a weight matrix W^O that was trained jointly with the model

X



3) The result would be the Z matrix that captures information from all the attention heads. We can send this forward to the FFNN

=



1) This is our input sentence*

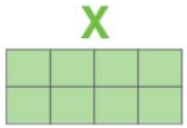
2) We embed each word*

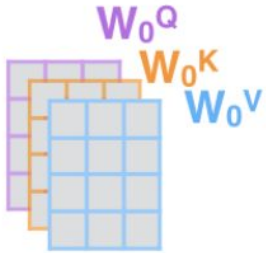
3) Split into 8 heads. We multiply X or R with weight matrices

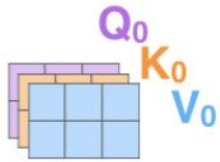
4) Calculate attention using the resulting Q/K/V matrices

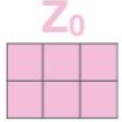
5) Concatenate the resulting Z matrices, then multiply with weight matrix W^O to produce the output of the layer

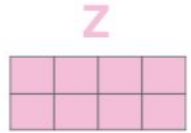
Thinking Machines



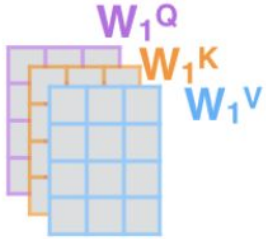


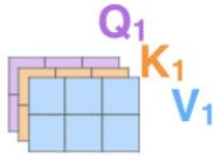


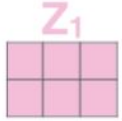


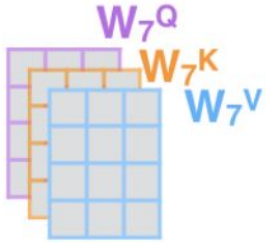


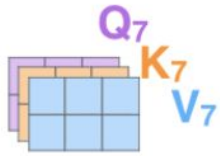
* In all encoders other than #0, we don't need embedding. We start directly with the output of the encoder right below this one

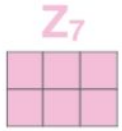




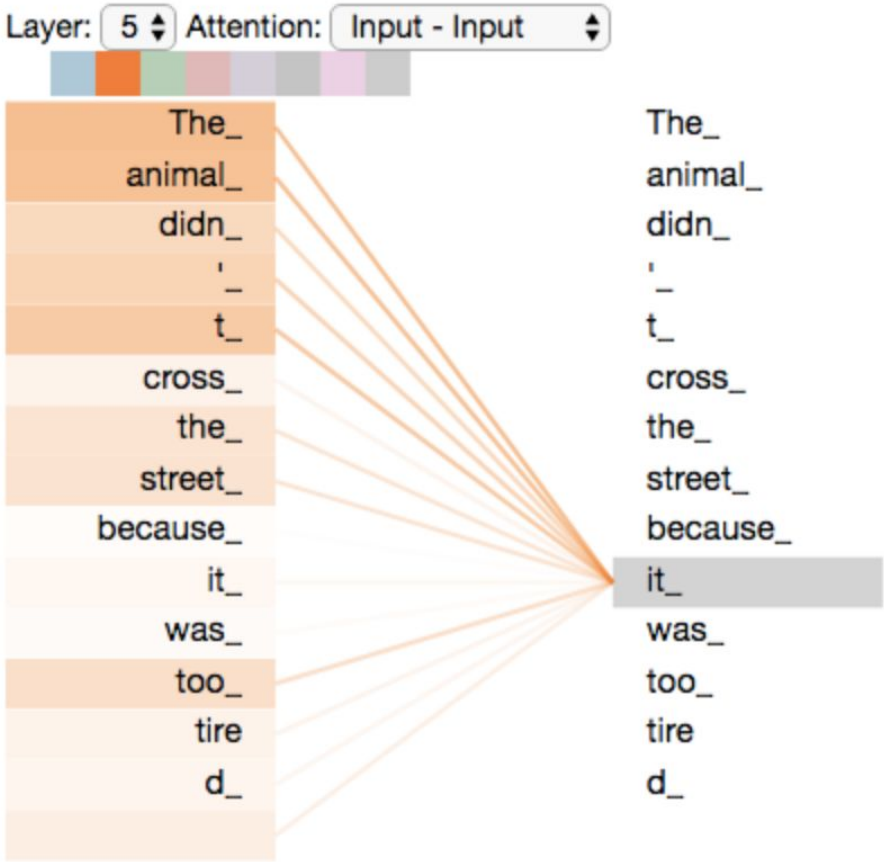


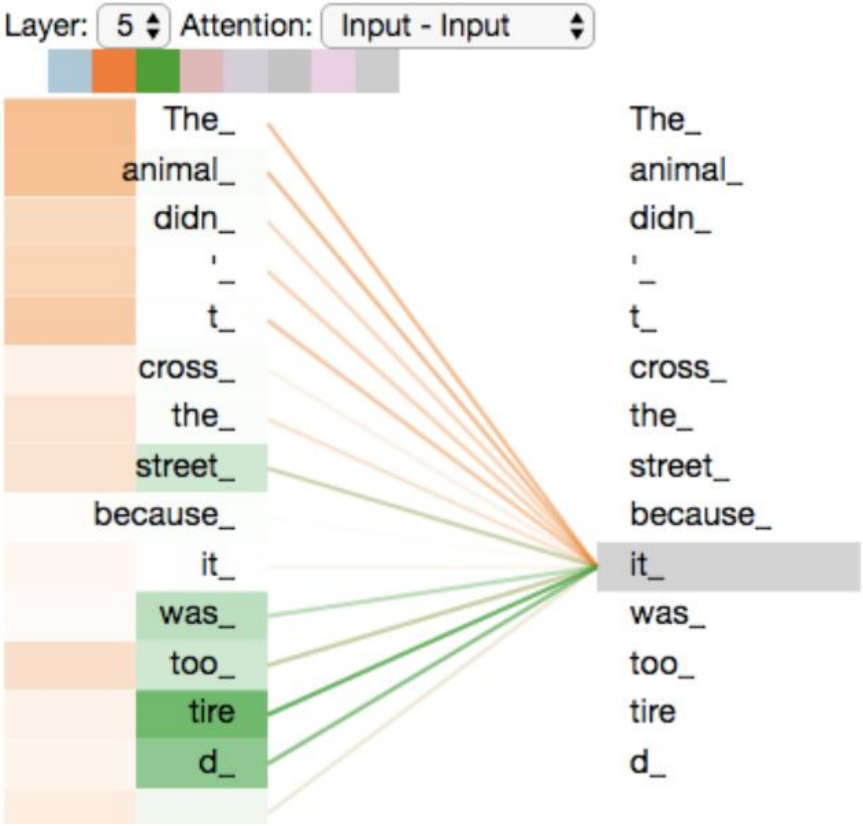






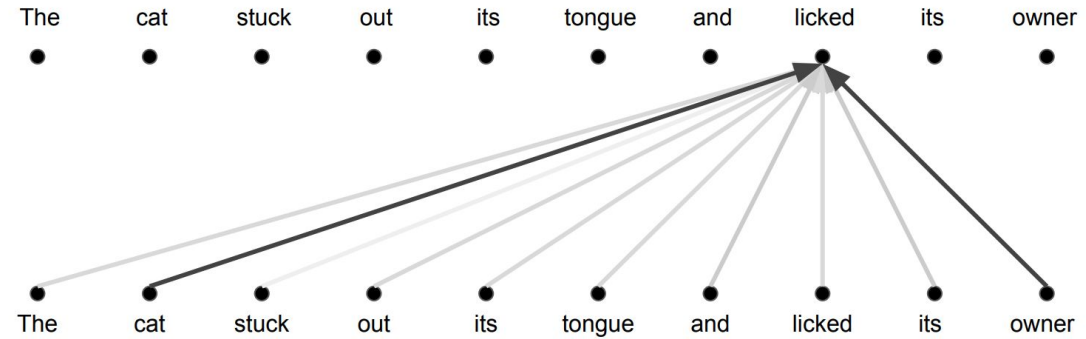
Multi-Head Attention





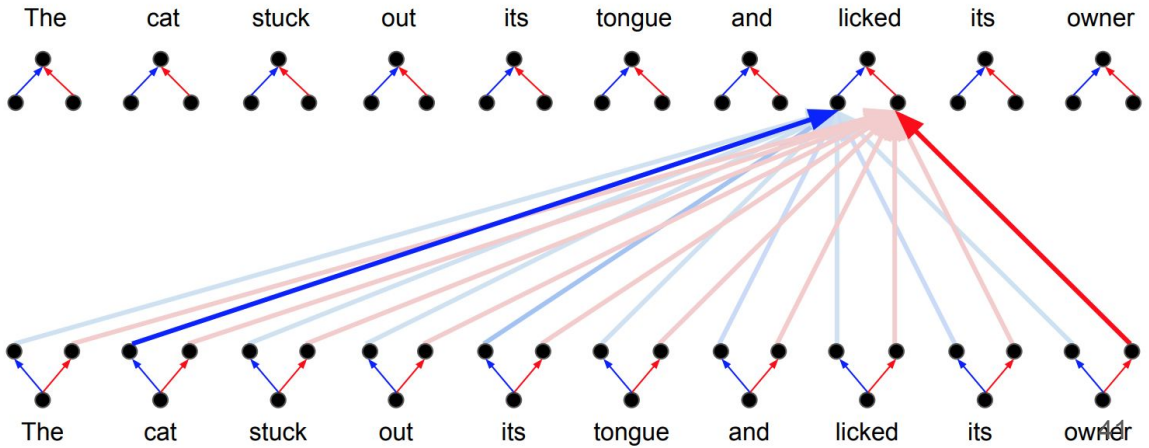
Attention vs. Multi-Head Attention

Attention: a weighted average



Multi-Head Attention:

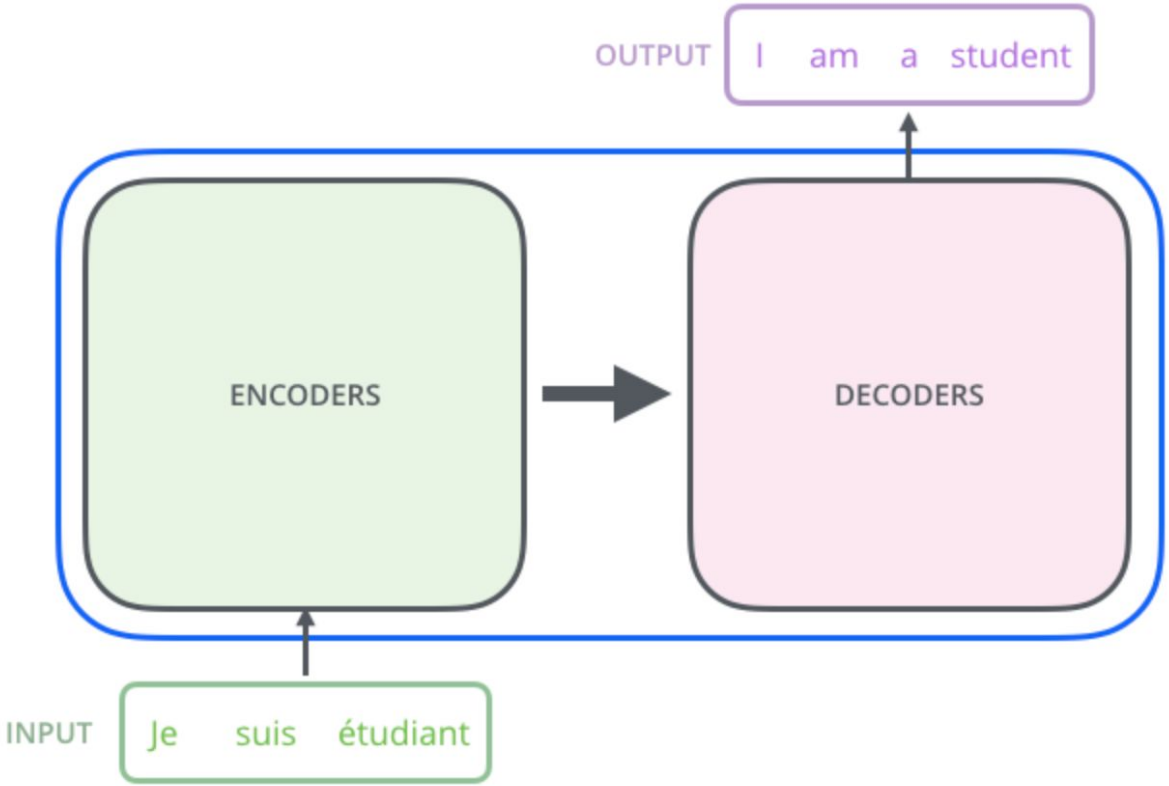
parallel attention layers with different linear transformations on input and output.



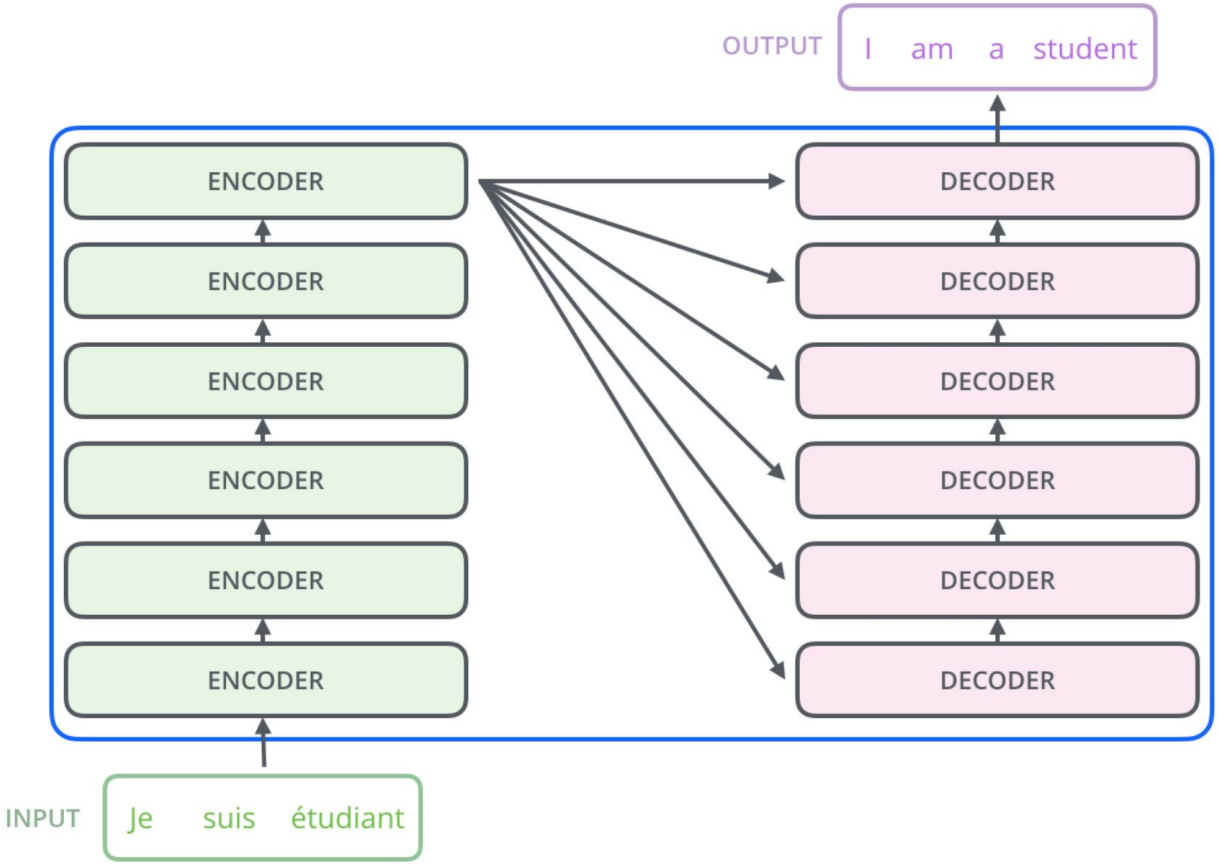
Transformer

The Transformer

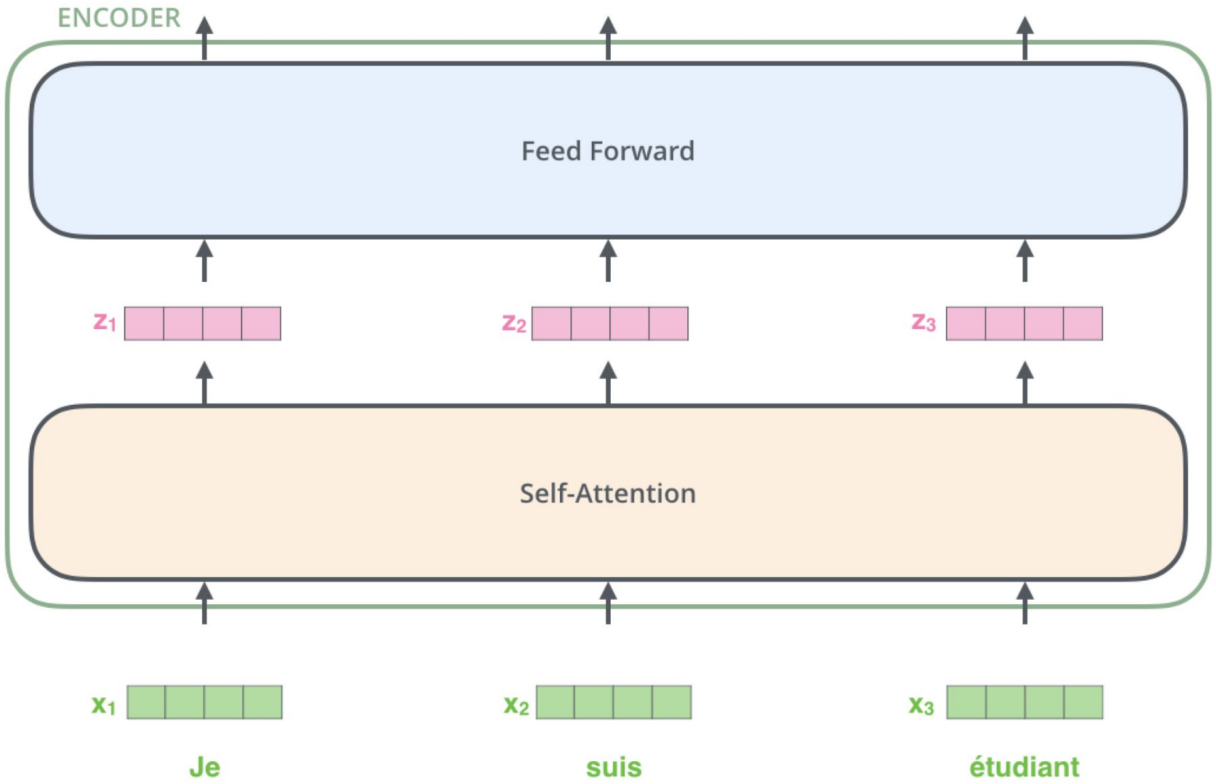
The Transformer



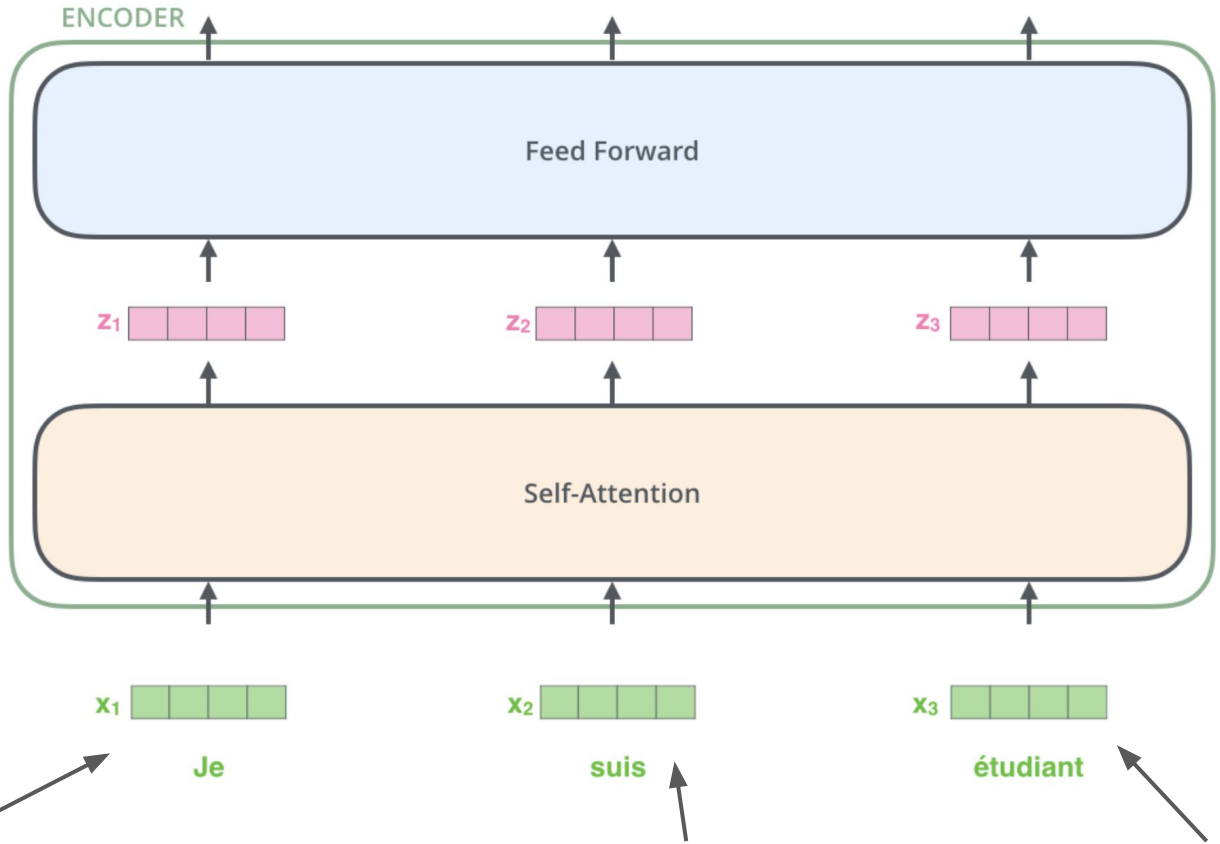
The Transformer



The Encoder Side



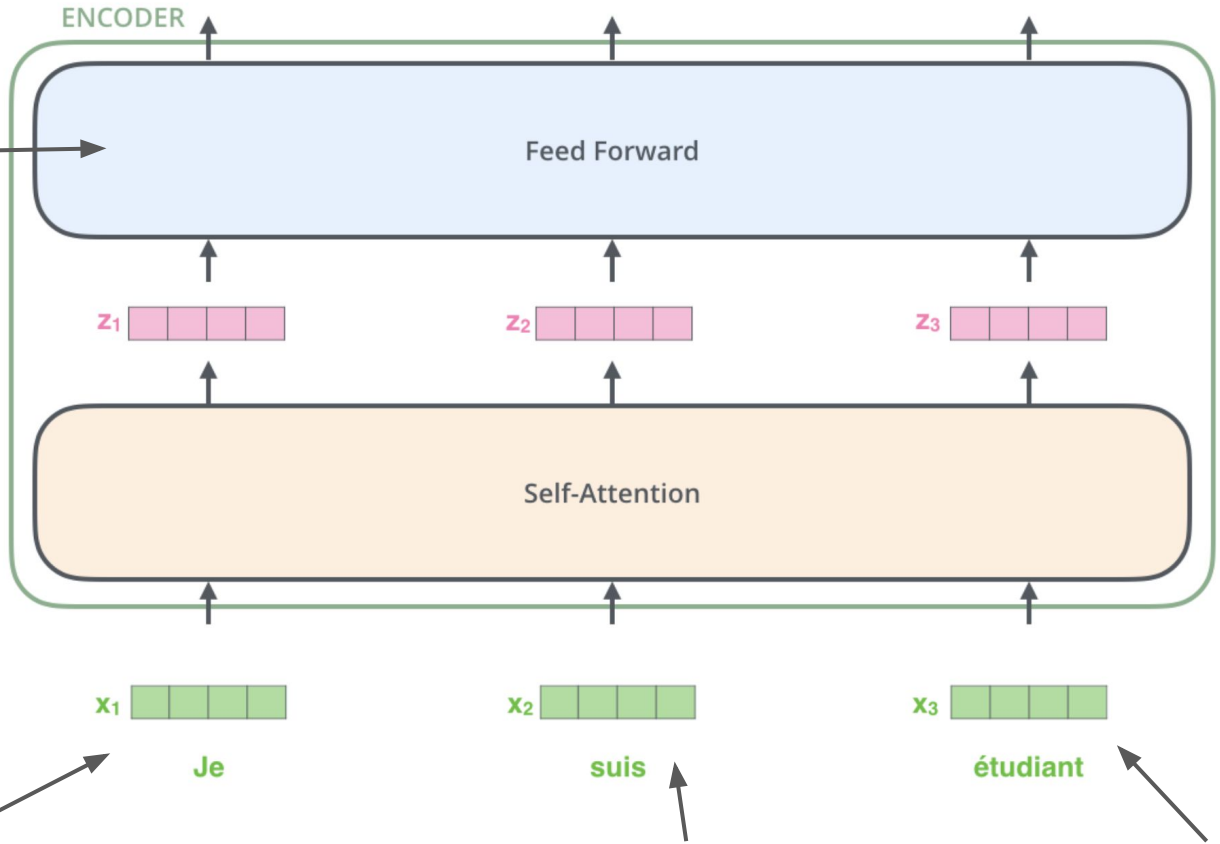
The Encoder Side



the word in each position flows through its own path in the encoder

The Encoder Side

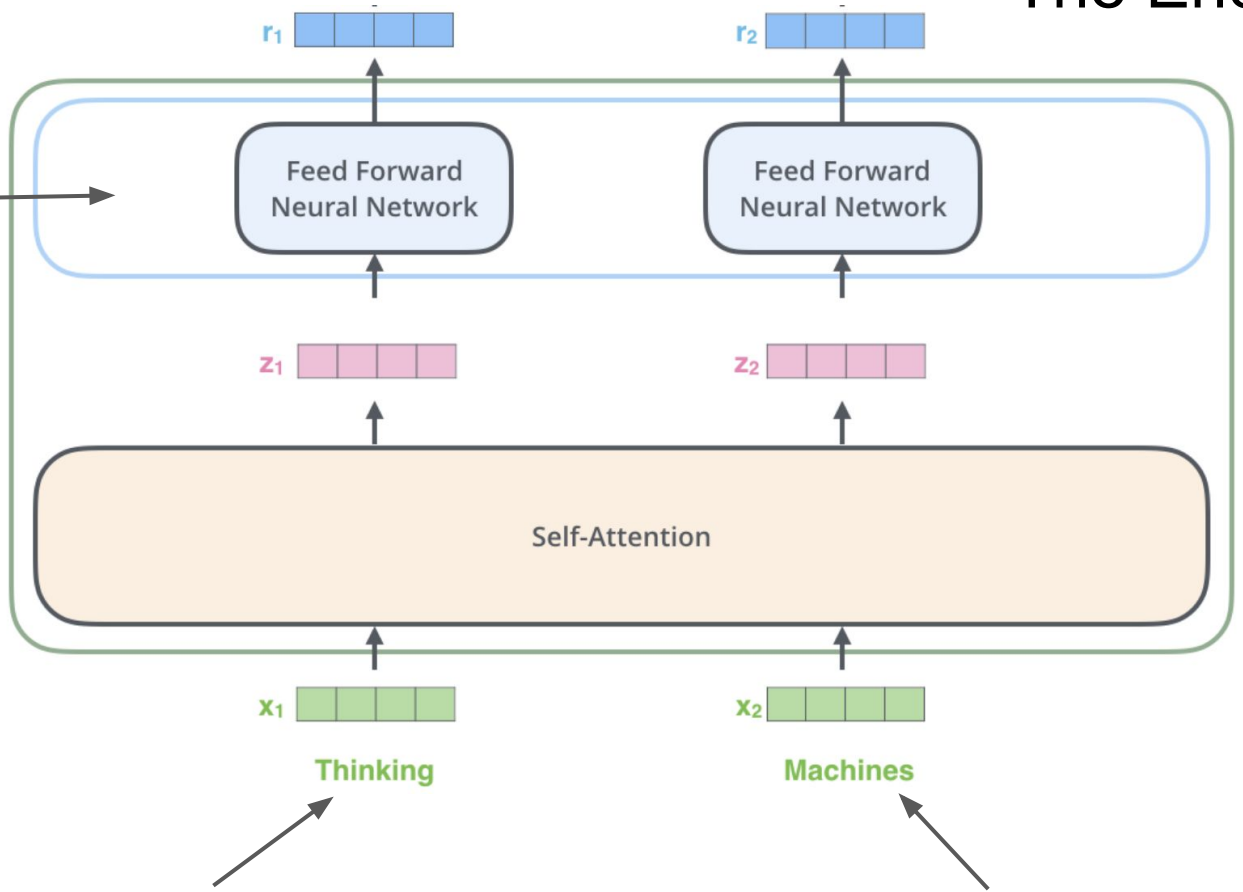
Can be parallelized



the word in each position flows through its own path in the encoder

The Encoder Side

Can be parallelized



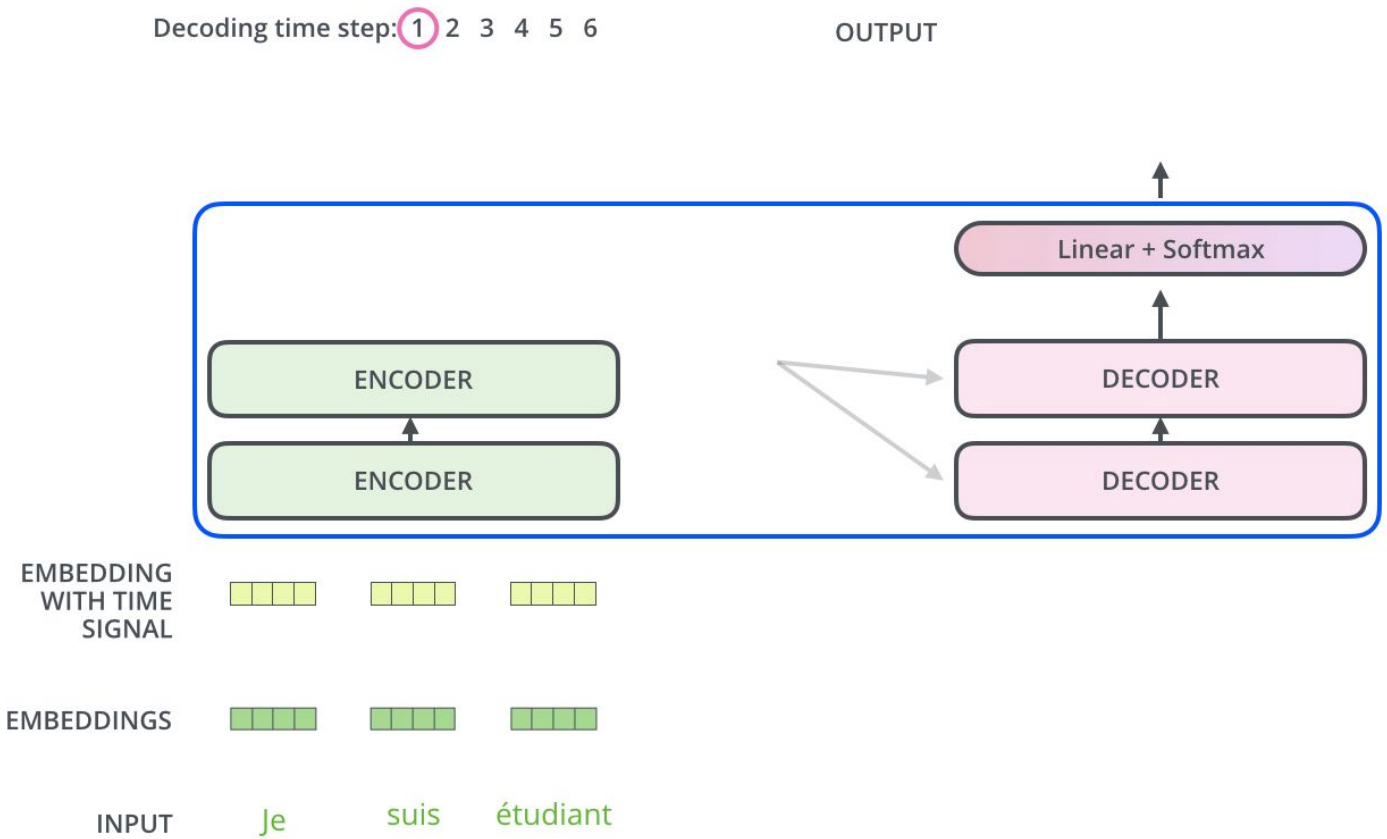
the word in each position flows through its own path in the encoder

The Transformer: quick overview

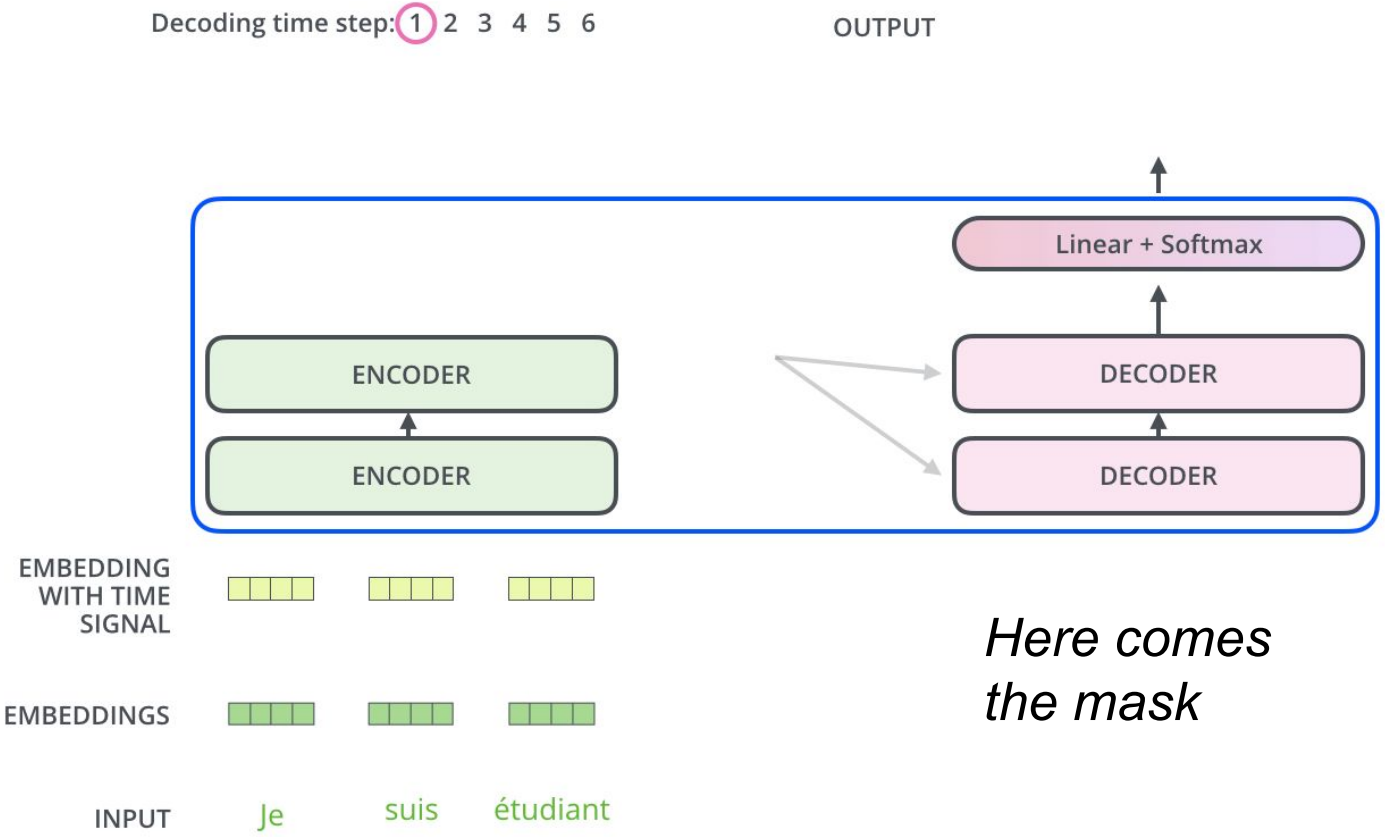
- Proposed in the paper “Attention is All You Need” (Ashish Vaswani et al.)
- No recurrent or convolutional neural networks -> just attention
- Beats seq2seq in machine translation task
  - 28.4 BLEU on the WMT 2014 English-to-German translation task
- Much faster
- Uses **self-attention** concept

The Decoder

The Decoder Side

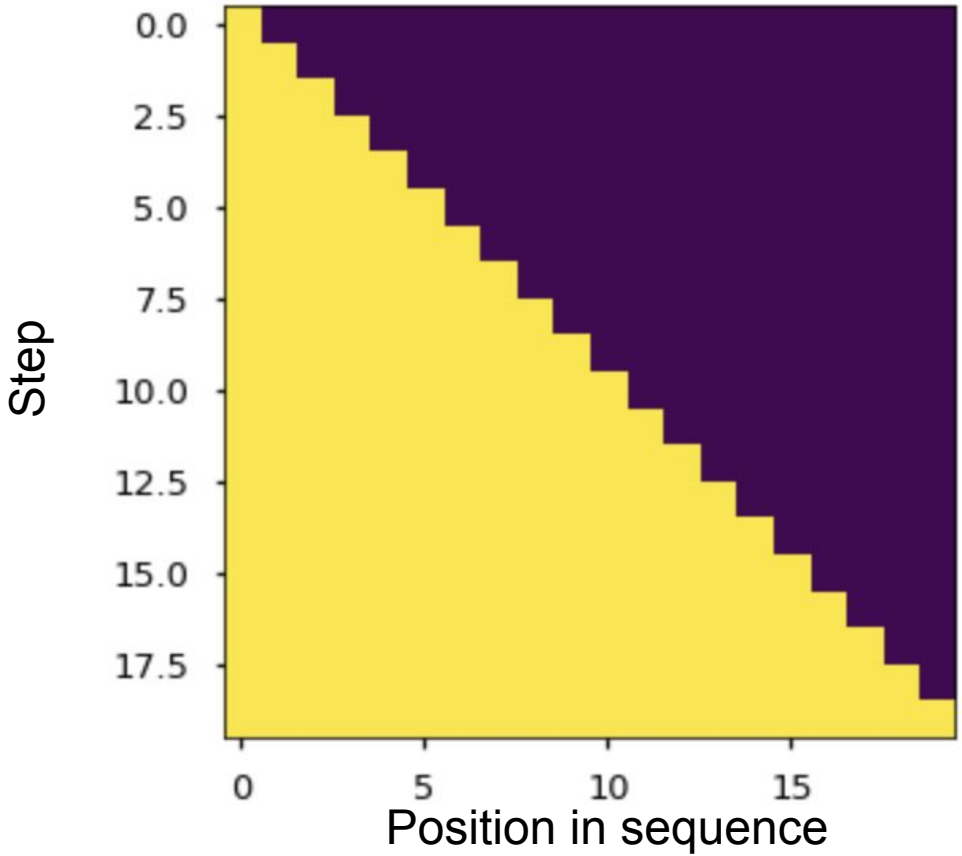


The Decoder Side

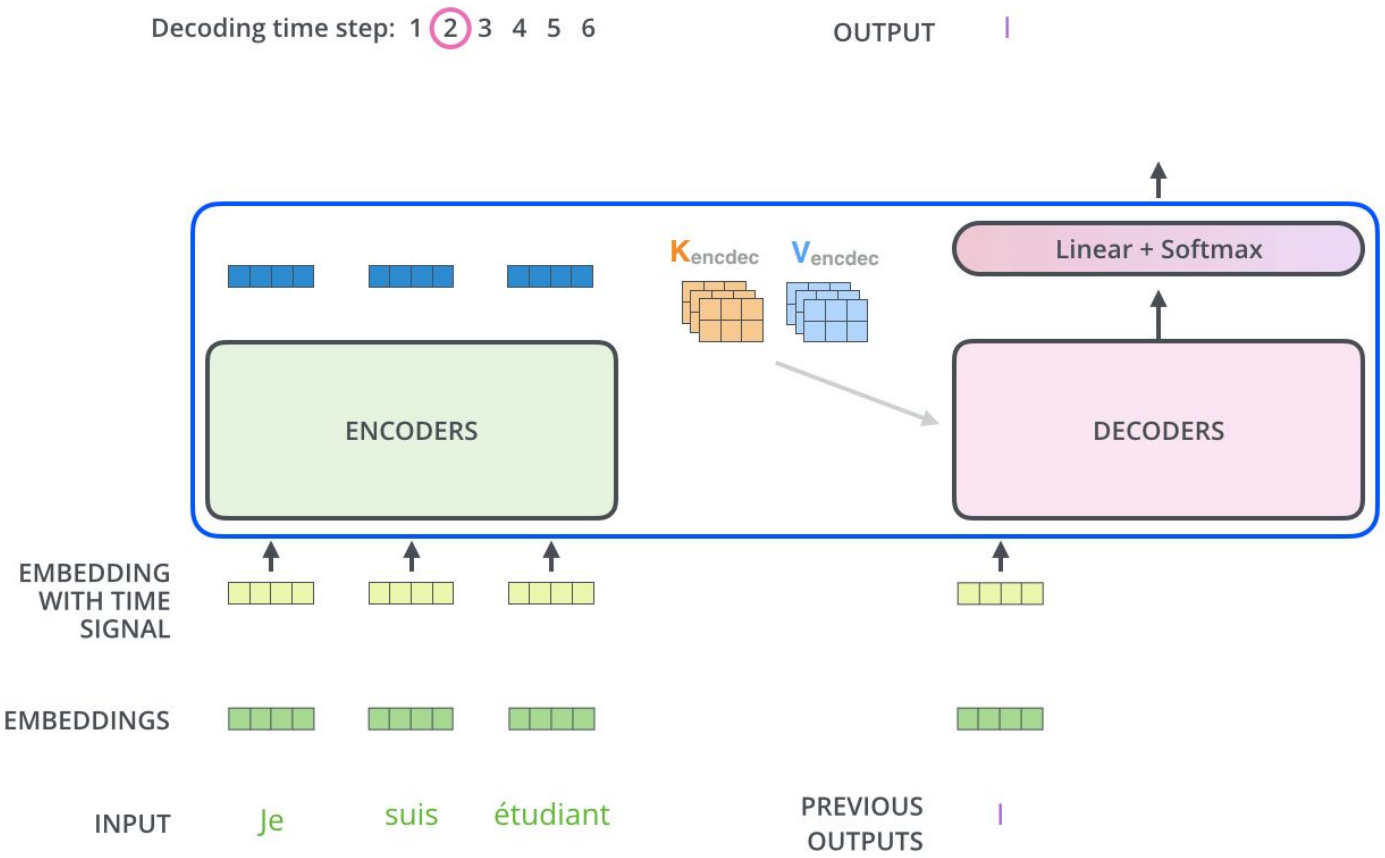


Here comes the mask

The masked decoder input

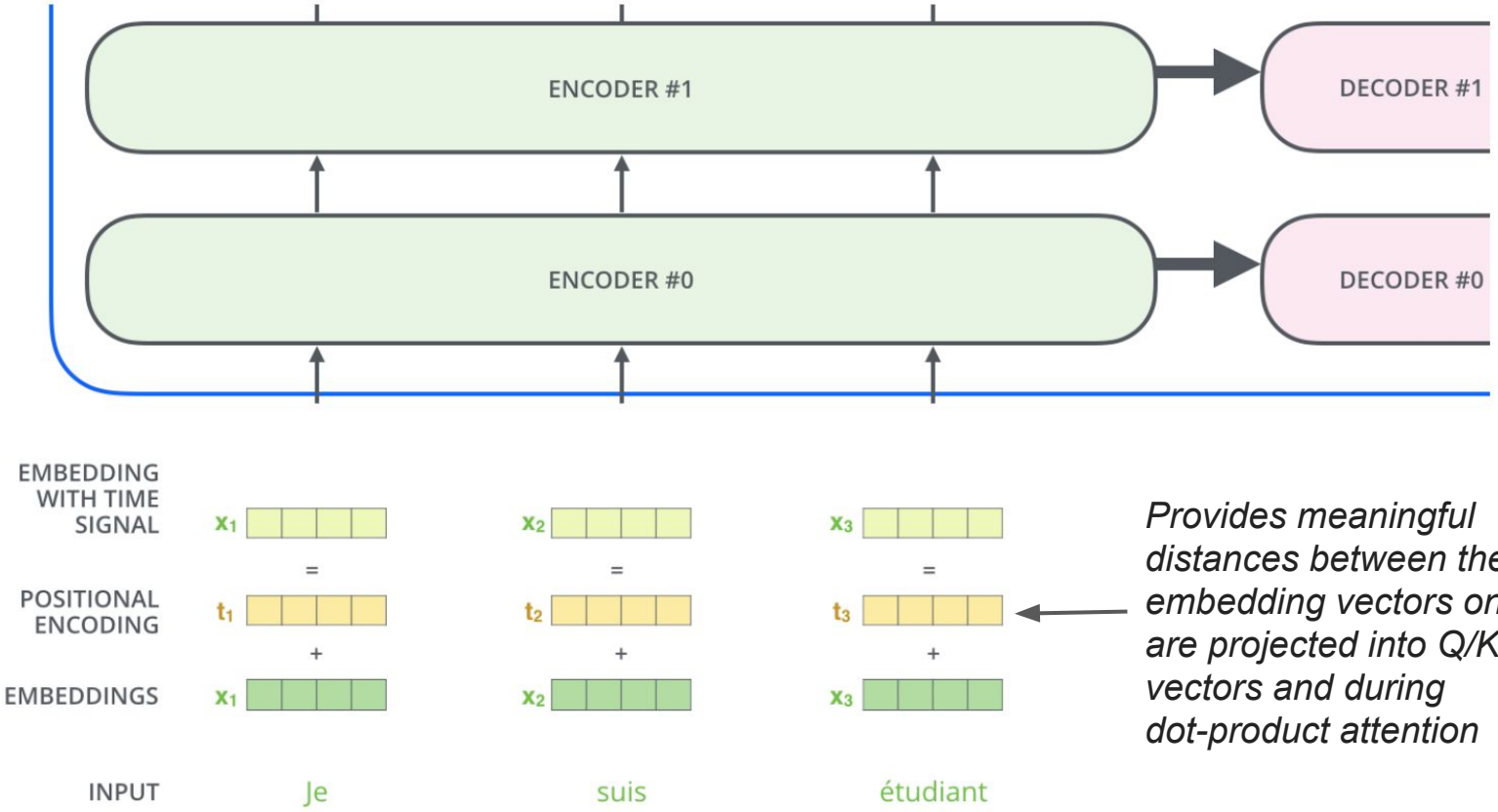


The Decoder Side



Positional Encoding

Positional Encoding



Provides meaningful distances between the embedding vectors once they are projected into Q/K/V vectors and during dot-product attention

Positional Encoding

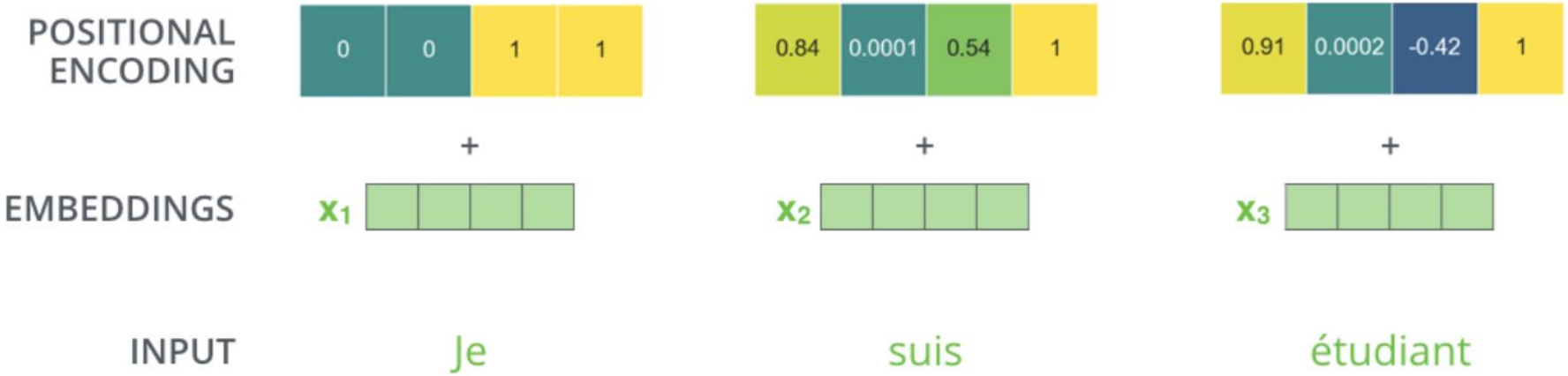
$$PE_{(pos,\ 2i)} = sin(pos / 10000^{2i\ /\ d_{\text{model}}})$$

$$PE_{(pos,\ 2i+1)} = cos(pos / 10000^{2i\ /\ d_{\text{model}}})$$

- *pos* is the position
- *i* is the dimension

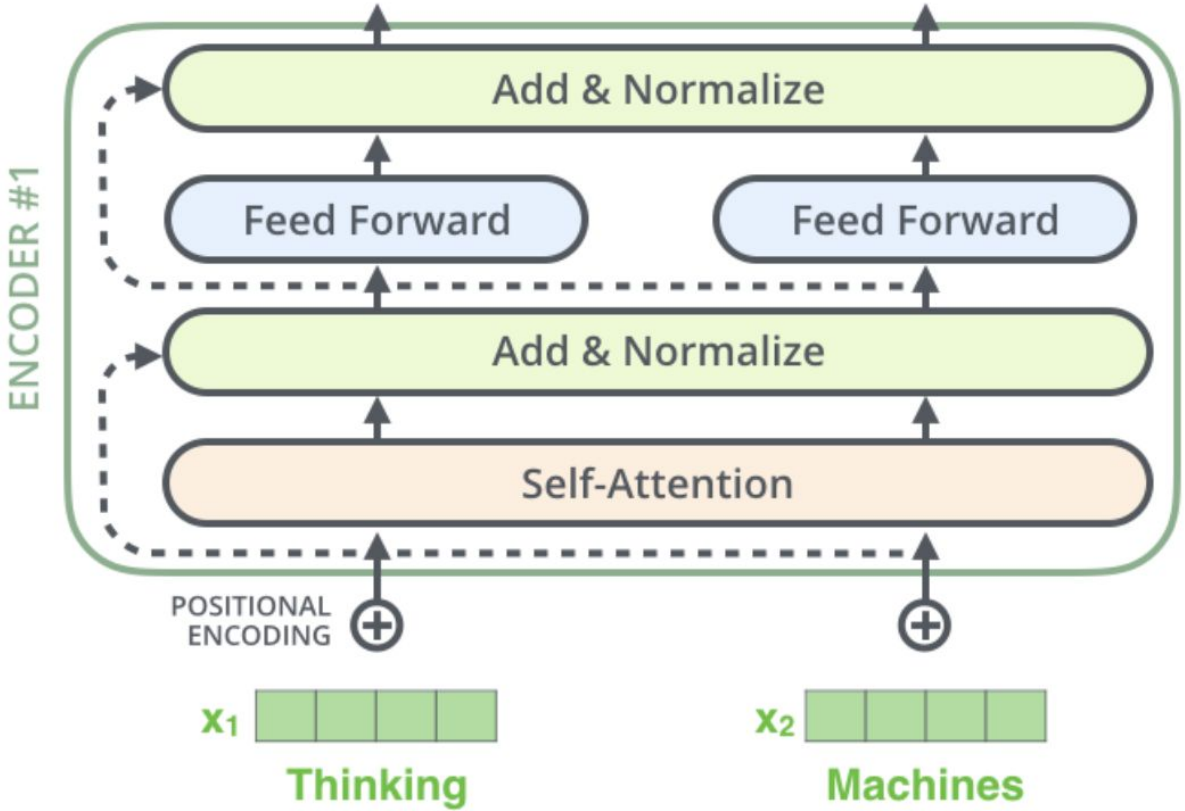
Each dimension of the positional encoding corresponds to a sinusoid.
The wavelengths form a geometric progression from 2pi to 2pi * 10 000

Positional Encoding



Layer Normalization

Layer Normalization

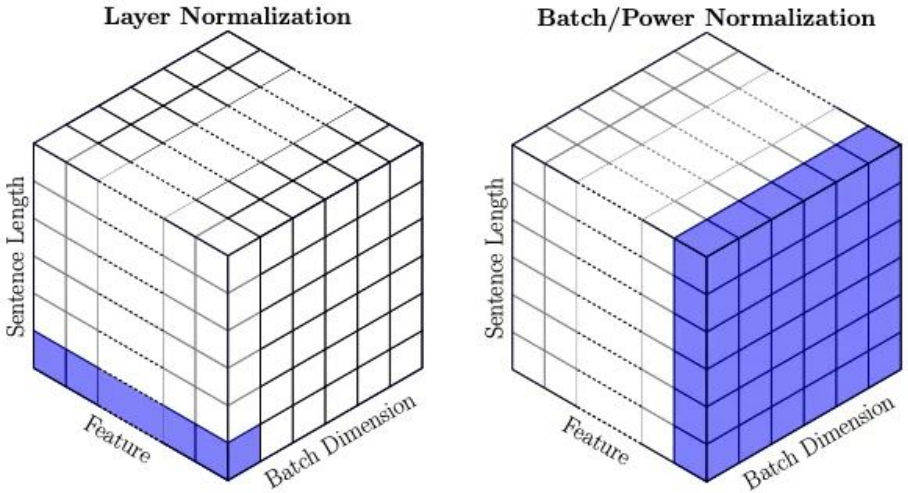


Layer Norm vs. Batch Norm

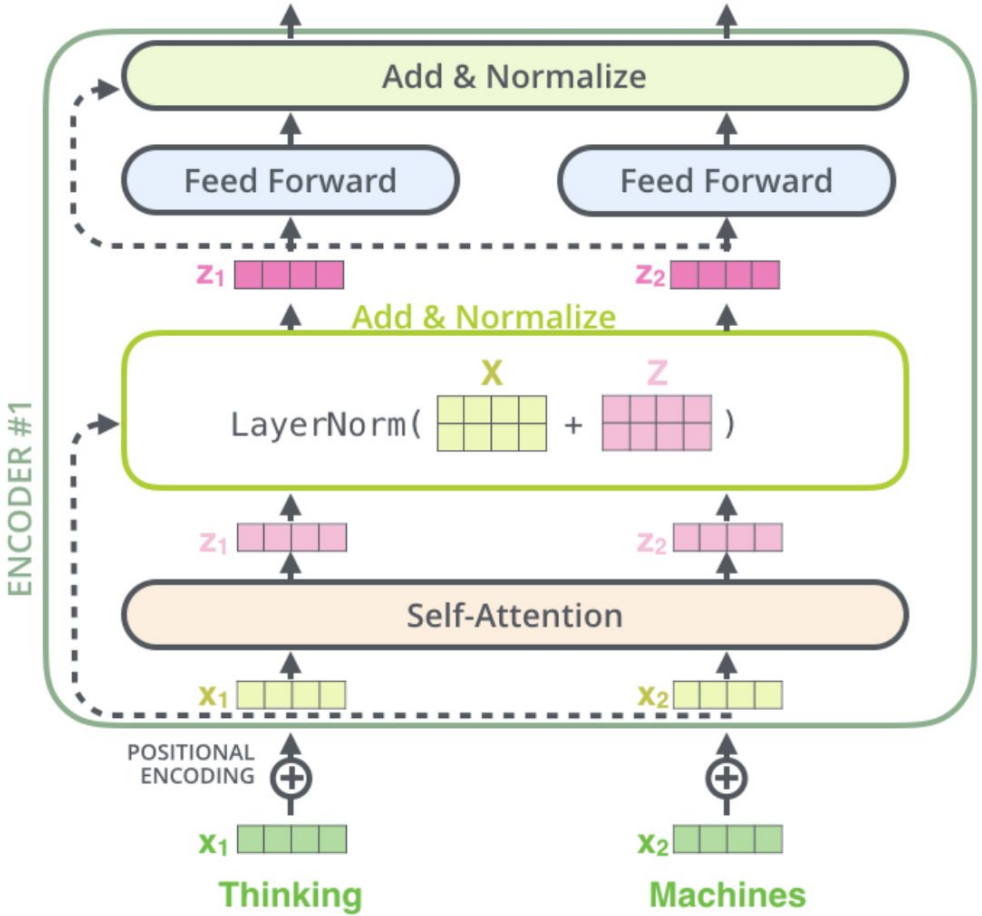
$$\mathrm{LN}(\mathbf{h}_i) = \frac{\mathbf{h}_i - \hat{\mu}_i}{\hat{\sigma}_i}$$

$$\hat{\mu}_i = \frac{1}{d}\sum_{j=1}^{d} \mathbf{h}_{ij}$$

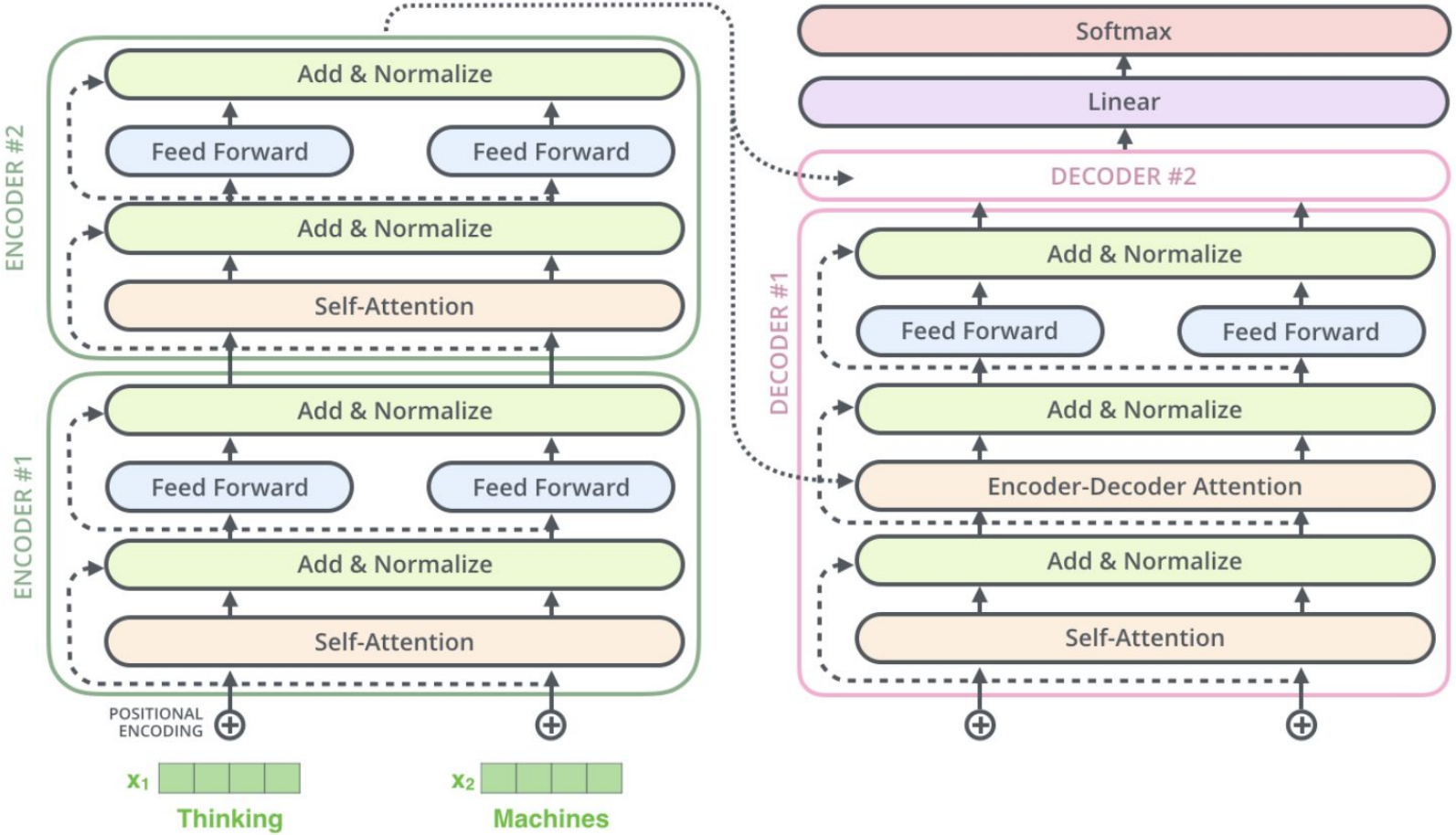
$$\hat{\sigma}_i = \sqrt{\frac{1}{d}\sum_{j=1}^{d} (\mathbf{h}_{ij} - \mu_i)^2}$$



Layer Normalization



Layer Normalization



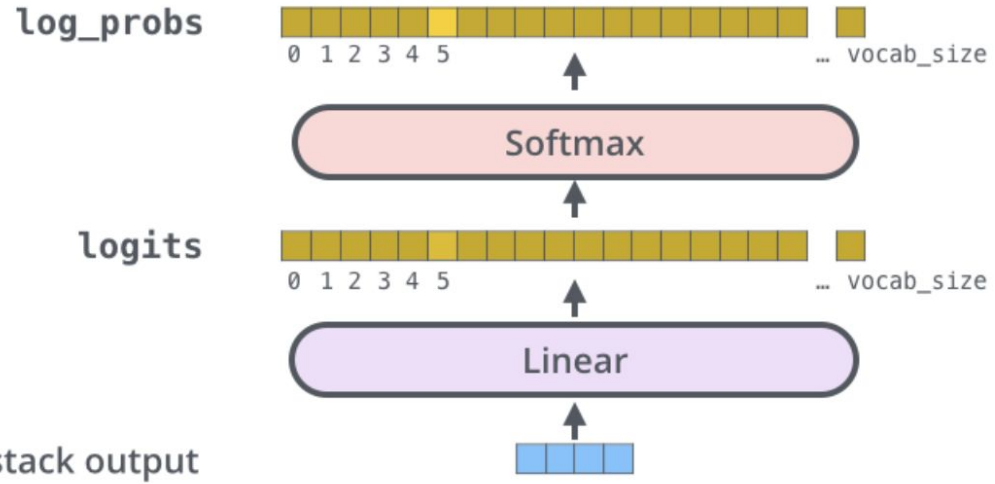
Final Linear and Softmax Layer

Which word in our vocabulary is associated with this index?

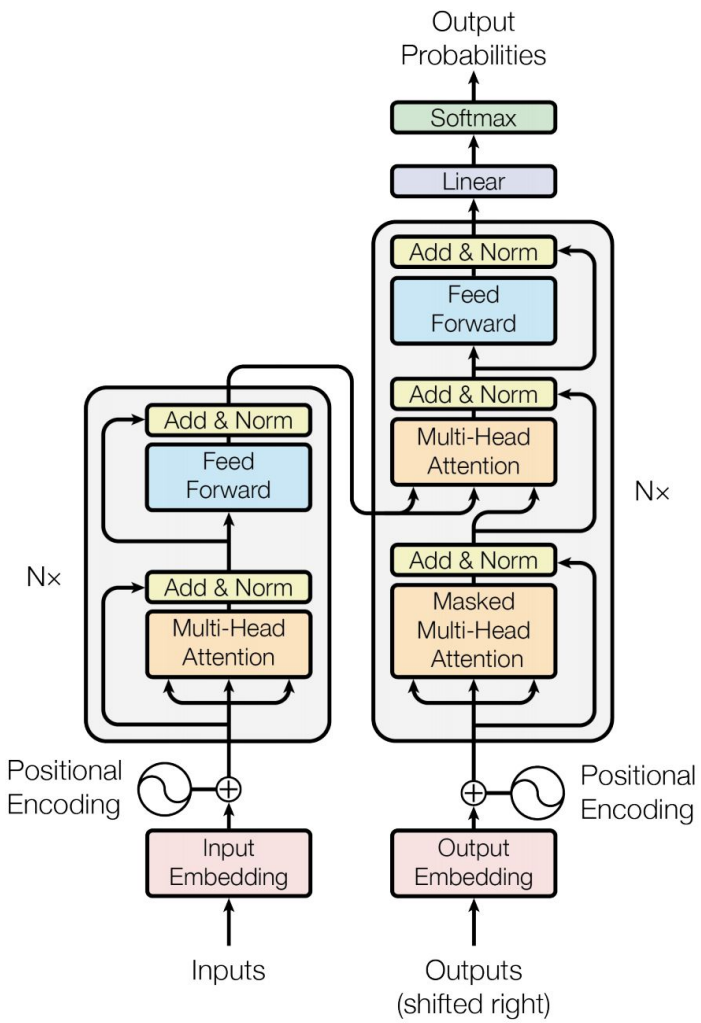
am

Get the index of the cell with the highest value **(argmax)**

5



The Transformer



Transformer Performance

Translation – WMT 2014 (BLEU)

	EN-DE	EN-FR
GNMT (orig)	24.6	39.9
ConvSeq2Seq	25.2	40.5
Transformer*	**28.4**	**41.8**

*Transformer models trained >3x faster than the others.

Q&A

Название раздела

natural-language-processing